

Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

До захисту допущено:
В.о. завідувача кафедри

_____ **Артем ВОЛОКИТА**
(підпис)
_____ 2026 р.
(число) (місяць)

Звіт з практики
на здобуття ступеня бакалавра
за освітньо-професійною програмою «Комп'ютерні системи та мережі»
спеціальність 123 — «Комп'ютерна інженерія»

**СИСТЕМА ПРОСТОРОВОГО АНАЛІЗУ ГРИ В ВОЛЕЙБОЛ З
ВИКОРИСТАННЯМ МАШИННОГО НАВЧАННЯ**

Виконав

Безщасний Роман Русланович, студент 4 курсу, група ІО-21 _____
(підпис)

Керівник

Морозов-Леонов Олександр Святославович, аспірант _____
(підпис)

Консультант

, _____
(підпис)

Рецензент

, _____
(підпис)

Засвідчую, що у цьому дипломному проєкті немає запозичень
з праць інших авторів без відповідних посилань

_____ **Безщасний Р.Р.**
(підпис)

Київ — 2026 р.

Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

Освітньо-кваліфікаційний рівень: Бакалавр

Освітньо-професійна програма: «Комп'ютерні системи та мережі»

Спеціальність: 123 — «Комп'ютерна інженерія»

ЗАТВЕРДЖУЮ

В.о. завідувача кафедри

Артем ВОЛОКИТА

_____ (підпис)

_____ (число)

_____ (місяць)

_____ 2026 р.

ЗАВДАННЯ

на бакалаврський дипломний проєкт

Безщасному Роману Руслановичу

- Тема проєкту: Система просторового аналізу гри в волейбол з використанням машинного навчання
керівник Морозов-Леонов Олександр Святославович, аспірант
затверджено наказом по університету від червня 2026 року №
- Дата здачі закінченого проєкту червня 2026 року
(число, місяць, рік)
- Вихідні дані для проєкту: технічна документація та статистичні дані.
- Зміст розрахунково-пояснювальної записки:
Розділ 1. Аналіз теорії та обґрунтування теми
Розділ 2. Математичне та алгоритмічне забезпечення аналітичної системи
Розділ 3. Програмна реалізація системи
Розділ 4. Аналіз результатів та експериментальні дослідження

5. Перелік графічного матеріалу: принципова, структурна та функціональна схеми системи.

6. Консультанти розділів проєкту

№	Розділ	Консультант (прізвище та ініціали)	Підпис, дата	
			завдання видав	завдання прийняв
1				

7. Дата видачі завдання 10 травня 2026 року

(число, місяць, рік)

Календарний план

№	Назва етапу виконання дипломного проєкту	Термін виконання	Примітка
1	Затвердження теми проєкту	06.04.2026–07.06.2026	
2	Аналіз предметної області	01.03.2026–30.04.2026	
3	Розробка архітектури та загальної структури системи	10.04.2026–10.05.2026	
4	Програмна реалізація системи	20.04.2026 - 25.05.2026	
5	Оформлення пояснювальної записки	18.05.2026-30.05.2026	
6	Передзахист		
7	Захист		

Виконавець _____ Безщасний Роман Русланович
(підпис)

Керівник _____ Морозов-Леонов Олександр Святославович
(підпис)

АНОТАЦІЯ

Об'єм і структура проєкту. Проєкт складається з вступу, чотирьох розділів, висновків, переліку умовних скорочень та списку літератури.

Актуальність теми. Роботу присвячено розробці програмно-алгоритмічного комплексу для автоматизованого відстеження волейбольного м'яча та розрахунку його кінематичних параметрів(швидкості, 3D-траєкторії) на основі монокулярного відеопотоку.

В процесі виконання роботи проаналізовано існуючі комерційні системи спортивної аналітики та виявлено потребу в доступних мобільних рішеннях. Розроблено архітектуру системи, що базується на моделі глибокого навчання YOLO для високоточної об'єктної детекції в умовах розмиття рухом. Для переходу від двовимірних піксельних координат до фізичних метричних величин імплементовано модель камери-обскури з використанням алгоритму просторового калібрування PnP та методу зворотного проектування.

Для вирішення проблеми високочастотного апаратного шуму та компенсації нелінійних аеродинамічних ефектів застосовано математичний апарат оптимальної фільтрації — сигма-точковий фільтр Калмана. З метою точного відстеження різких змін швидкості під час атакуючих ударів розроблено гібридну архітектуру на базі взаємодіючих множинних моделей.

Експериментальне тестування підтвердило здатність системи реконструювати тривимірну траєкторію та фіксувати пікову швидкість удару з використанням однієї камери стандартної частоти кадрів (50-60 FPS). Результати роботи можуть бути використані тренерськими штабами для тактико-технічного аналізу та оптимізації тренувального процесу у волейболі.

Ключові слова: *модель глибокого навчання YOLO, алгоритм просторового калібрування PnP, метод зворотного проектування, нелінійні аеродинамічні ефекти, сигма-точковий фільтр Калмана, взаємодіючі множинні моделі, тривимірна траєкторія, пікова швидкість удару.*

ANNOTATION

The thesis is devoted to the development of a software and algorithmic complex for automated volleyball tracking and calculation of its kinematic parameters (speed, 3D trajectory) based on a monocular video stream.

During the research, existing commercial sports analytics systems were analyzed, revealing a strong demand for accessible mobile solutions. The system architecture is based on the YOLO deep learning model for high-precision object detection under severe motion blur conditions. To transition from 2D pixel coordinates to physical metric values, a pinhole camera model was implemented using the Perspective-n-Point(PnP) spatial calibration algorithm and Ray Casting method.

To solve the problem of high-frequency hardware noise and compensate for non-linear aerodynamic effects, the mathematical apparatus of optimal filtering was applied, specifically the Unscented Kalman Filter. In order to accurately track abrupt speed changes during spikes, a hybrid architecture based on Interacting Multiple Models was designed.

Experimental testing confirmed the system's ability to reconstruct the 3D trajectory and capture the peak spike speed using a single standard frame rate camera (50-60 FPS). The results of the work can be utilized by coaching staffs for tactical and technical analysis and the optimization of the training process in volleyball.

Keywords: *YOLO deep learning model, PnP spatial calibration algorithm, Ray Casting method, nonlinear aerodynamic effects, Unscented Kalman Filter, Interacting Multiple Models three-dimensional, trajectory, peak impact velocity.*

[illegible]

**СИСТЕМА ПРОСТОРОВОГО АНАЛІЗУ ГРИ В ВОЛЕЙБОЛ З
ВИКОРИСТАННЯМ МАШИННОГО НАВЧАННЯ**

Технічне завдання

ІАЛЦ.467200.002 ТЗ

ЗМІСТ

1. НАЙМЕНУВАННЯ ТА ОБЛАСТЬ ЗАСТОСУВАННЯ	2
2. ПІДСТАВИ ДЛЯ РОЗРОБКИ	2
3. МЕТА ТА ПРИЗНАЧЕННЯ РОЗРОБКИ	2
4. ДЖЕРЕЛА РОЗРОБКИ	3
5. ТЕХНІЧНІ ВИМОГИ	3
5.1. Вимоги до вхідних даних	3
5.2. Вимоги до функціональних модулів	3
5.3. Вимоги до апаратної частини	4
6. ЕТАПИ РОЗРОБКИ	4
6.1 Аналітичний етап	4
6.2 Нейромережевий етап	5
6.3 Математичний етап	5
6.4 Інтеграційний етап	5
6.5 Документаційний етап	5

					ІАЛЦ.467200.002 ТЗ			
Зм.	Лист	№ докум.	Підп.	Дата				
Розробив	Безщасний Р.Р.				Система просторового аналізу гри в волейбол з використанням машинного навчання		Лит.	Аркуш
Перевірів	Морозов-Леонів							Аркуші
								1
								5
Н. контр.					НТУУ "КПІ ім. Ігоря Сікорського", ФІОТ ІО-21			
Затвердив	Волокита А. М.							

1. НАЙМЕНУВАННЯ ТА ОБЛАСТЬ ЗАСТОСУВАННЯ

В цьому технічному завданні охоплено опис розробки та впровадження програмно-алгоритмічного комплексу комп'ютерного зору для аналітики волейбольних подій, завдяки аналізу позиції м'яча в тривимірному просторі

Областю застосування цієї системи є спортивна аналітика, автоматизований збір статистики тренувальних та змагальних процесів у волейболі, а також інструментарій для тактико-технічного аналізу та оцінки кінематики рухів гравців тренерськими штабами аматорських і професійних ліг.

2. ПІДСТАВИ ДЛЯ РОЗРОБКИ

Першою підставою для розробки є завдання на виконання бакалаврської кваліфікаційної роботи рівня «бакалавр комп'ютерної інженерії», який був затверджений факультетом "Інформатики та обчислювальної техніки" кафедрою обчислювальної техніки Національного технічного Університету України «Київський Політехнічний інститут ім. Ігоря Сікорського».

А другою підставою є необхідність створення актуальної мобільної та обчислювально доступної альтернативи дорогим багатокамерним стаціонарним комплексам, що здатна функціонувати на основі монокулярних відеопотоків із побутових камер.

3. МЕТА ТА ПРИЗНАЧЕННЯ РОЗРОБКИ

Метою розробки є створення високоточної системи для розрахунку кінематичних метрик волейбольного м'яча(тривимірної траєкторії, пікової швидкості спайку, точок відскоку) з усуненням апаратних шумів та впливу розмиття рухом.

Призначенням розробки є програмний комплекс для автоматизації процесу перетворення сирих двовимірних відеоданих у фізичну метричну си-

стему тривимірного простору з використанням сучасних методів глибокого навчання та ймовірнісного нелінійного згладжування.

4. ДЖЕРЕЛА РОЗРОБКИ

Джерелами розробки є

- наукова й технічна література з методів комп’ютерного зору, перспективної просторової геометрії, багатокритерійного трекінгу та теорії оптимального оцінювання станів динамічних систем;
- Офіційна документація алгоритмів об’єктної детекції архітектури YOLO;
- Відкриті програмні бібліотеки та фреймворки (OpenCV, Ultralytics, FilterPy, NumPy, PyTorch).

5. ТЕХНІЧНІ ВИМОГИ

5.1. Вимоги до входних даних

- Система повинна обробляти відеопотоки стандартних форматів (MP4, AVI) з роздільною здатністю не менше 1920x1080 пікселів.
- Мінімально допустима частота кадрів для коректного розрахунку динаміки – 50 FPS(з використанням методів програмного вирівнювання VFR у CFR).

5.2. Вимоги до функціональних модулів

- *Модуль детекції:* Використання anchor-free моделі YOLO(рівня yolo26m) зі збільшеним розміром входного тензора (imgsz=1088) для детекції дрібних об’єктів.

					ІАЛЦ.467200.002 ТЗ	Аркуш
Зм.	Лист	№ докум.	Підп.	Дата		3

- *Геометричний модуль*: Реалізація перетворення 2D-пікселів у 3D-промені(Ray Casting) на основі алгоритму просторового калібрування PnP(solvePnP) із механізмом інверсії координатних осей під балістичну модель.
- *Модуль трекінгу*: Адаптований алгоритм 3D ByteTrack з двостадійною асоціацією даних та використанням статистичної відстані Махаланобіса замість IoU.
- *Модуль фільтрації*: Нелінійне згладжування траєкторій за допомогою багатомодельної архітектури IMMу поєднанні з сигматочковими фільтрами Калмана (UKF). Наявність захисту від обчислювальних аномалій(вибух коваріації).

5.3. Вимоги до апаратної частини

- Операційна система: Windows 10/11 або дистрибутиви Linux(Ubuntu).
- Мова програмування: Python версії 3.11 або вище.
- Апаратна частина: багатоядерний центральний процесор (CPU) та графічний прискорювач (GPU) із підтримкою технології паралельних обчислень CUDA для забезпечення мілісекундного інференсу нейромережі.

6. ЕТАПИ РОЗРОБКИ

6.1 Аналітичний етап

Вивчення предметної області, збір власних відеоданих, їх попереднє оброблення та підготовка датасету.

					ІАЛЦ.467200.002 ТЗ	Аркуш
						4
Зм.	Лист	№ докум.	Підп.	Дата		

6.2 Нейромережевий етап

Проектування архітектури пайплайну, донавчання та валідація моделі YOLO на хмарних обчислювальних потужностях.

6.3 Математичний етап

Програмна реалізація алгоритмів Ray Casting, конфігурація матриць шуму та переходу для моделей IMM-UKF, розробка просторового трекера 3D ByteTrack.

6.4 Інтеграційний етап

Об'єднання розроблених модулів у єдиний конвеєр, проведення експериментів на реальних ігрових епізодах, отримання кінематичної статистики.

6.5 Документаційний етап

Оформлення вихідного коду, візуалізація графіків результатів та написання тексту пояснювальної записки.

					ІАЛЦ.467200.002 ТЗ	Аркуш
						5
Зм.	Лист	№ докум.	Підп.	Дата		

**СИСТЕМА ПРОСТОРОВОГО АНАЛІЗУ ГРИ В ВОЛЕЙБОЛ З
ВИКОРИСТАННЯМ МАШИННОГО НАВЧАННЯ**

Пояснювальна записка

ІА/Ц.467200.003 ПЗ

ЗМІСТ

Перелік умовних скорочень	3
Вступ	4
1 Аналіз теорії та обґрунтування теми	8
1.1 Еволюція методів виявлення рухомих об'єктів у відеопотоці .	8
1.2 Огляд та аналіз еволюції архітектур сімейства YOLO	11
1.3 Обмеження проєкцій двовимірного простору та математичне обґрунтування переходу до 3D	14
1.4 Проблема багатокритерійного трекінгу у 3D просторі	17
1.5 Порівняння методів фільтрації нелінійних динамічних систем	19
1.6 Особливості предметної області волейболу як цілі трекінгу . .	21
2 Математичне та алгоритмічне забезпечення аналітичної системи	28
2.1 Структура та граф взаємодії компонентів програмно-алгоритмічного конвеєра	28
2.2 Математична модель геометричного зворотного проєктування та вирівнювання базису осей	29
2.3 Математичний апарат UKF та нелінійної аеродинаміки	32
2.4 Налаштування матриць коваріації процесу та вимірювань . . .	35
2.5 Адаптація алгоритму ByteTrack для тривимірного простору . .	39
2.6 Фізична модель відскоку від поверхні підлоги	41
2.7 Методологія валідації якості трекінгу	43

					ІАЛЦ.467200.003 ПЗ			
<i>Зм.</i>	<i>Лист</i>	<i>№ докум.</i>	<i>Підп.</i>	<i>Дата</i>	Система просторового аналізу гри в волейбол з використанням машинного навчання	<i>Лит.</i>	<i>Аркуш</i>	<i>Аркушів</i>
Розробив	Безщасний Р.Р.						1	62
Перевірів	Морозов-Леонів							
Н. контр.						НТУУ "КПІ ім. Ігоря Сікорського", ФІОТ ІО-21		
Затвердив	Волокита А. М.							

Висновки до розділу 2	45
3 Програмна реалізація аналітичного комплексу	48
3.1 Загальна структура програмного комплексу	48
3.2 Модуль геометричного зворотного проектування	50
3.3 Модуль ядра фільтрації IMM_UKF	51
3.4 Модуль трекінгу IMMTracker	54
3.5 Модуль навчання моделі YOLO26	56
3.6 Конфігураційні параметри та залежності	57
3.7 Інструкція з користування програмним комплексом	59

ПЕРЕЛІК УМОВНИХ СКОРОЧЕНЬ

CNN	(Convolutional Neural Network) Згорткова нейронна мережа
EKF	(Extended Kalman Filter) Розширений фільтр Калмана
Faster R-CNN	(Faster Region-based Convolutional Neural Network) Швидша згорткова нейронна мережа на основі регіонів
IMM	(Interacting Multiple Model) Взаємодіюча множинна модель
IoU	(Intersection over Union) Перетин над об'єднанням
MoG2	(Mixture of Gaussians 2) Суміш гауссових коефіцієнтів 2
PnP	(Perspective-n-Point) Перспектива n-ої точки
RPN	(Region Proposal Network) Мережа пропозиції регіонів
SORT	(Simple Online and Realtime Tracking) Просте онлайн-відстеження та відстеження в режимі реального часу
UKF	(Unscented Kalman Filter) Сигма-точковий фільтр Калмана
YOLO	(You Only Look Once) Ти дивишся тільки раз

ВСТУП

Актуальність теми. Сучасний етап розвитку професійного спорту характеризується тотальною цифровізацією та інтеграцією інтелектуальних технологій у тренувальний та змагальний процеси. У волейболі, який є однією з найдинамічніших командних ігор у світі, швидкість польоту м'яча при атакуючих ударах може перевищувати 130 км/год, а тривалість окремих ігрових фаз вимірюється частками секунди. За таких умов традиційний візуальний аналіз з боку тренерського штабу або використання простих систем двовимірної відеофіксації вже не задовольняють вимогам високоточного розбору тактико-технічних дій.

Автоматизований збір кінематичних метрик, таких як тривимірна траєкторія польоту м'яча, координати точок відскоку, кути атаки та пікова швидкість польоту м'яча, дозволяє об'єктивно оцінювати ефективність дій гравців, моделювати поведінку суперника та оптимізувати біомеханіку рухів під час тренувань. Проте чинні комерційні рішення (наприклад, Hawk-Eye або Foxtenn) є монопольними системами з високою вартістю впровадження, що вимагають встановлення від 10 до 20 синхронізованих високошвидкісних індустріальних камер по всьому периметру залу. Це робить їх абсолютно недоступними для аматорських ліг, дитячо-юнацьких спортивних шкіл та клубів середнього рівня.

У зв'язку з цим виникає гостра потреба в розробці доступної, локальної та архітектурно гнучкої системи комп'ютерного зору, здатної здійснювати точний тривимірний трекінг та аналітику ігрових параметрів волейболу на основі обмеженої кількості джерел відеоданих (аж до однієї камери споживацького класу, наприклад, сучасного смартфона). Дана задача лежить на перетині глибокого навчання (для детекції дрібних швидкісних об'єктів), проєктивної геометрії (для калібрування простору) та теорії оптимальної філь-

трації й оцінювання станів(для компенсації шумів вимірювання). Реалізація такої системи дозволить створити оптимізований програмно-алгоритмічний пайплайн, що ефективно поєднує низькорівневу обробку відеопотоків із сучасними методами Data Science.

Мета дослідження. Метою бакалаврської роботи є розробка та дослідження системи комп'ютерного зору для високоточного автоматизованого аналізу волейбольних епізодів у тривимірному просторі за допомогою однієї статичної камери на основі моделей глибокого навчання та нелінійного адаптивного фільтрування.

Для досягнення поставленої мети необхідно вирішити такі завдання:

- проаналізувати існуючі методи детекції, трекінгу та відновлення 3D-координат швидкісних об'єктів за монокулярними відеопотоками;
- обґрунтувати вибір та провести донавчання згорткової нейронної мережі сімейства YOLO для стабільної детекції волейбольного м'яча в умовах сильного розмиття рухом;
- розробити програмний модуль геометричного калібрування тривимірному простору майданчика на основі математичного апарату перспективного проектування камери(PnP);
- створити алгоритм зворотного проектування для побудови 3D-променів від оптичного центру лінзи через піксельні координати детекції та оцінки глибини польоту м'яча;
- дослідити проблему високочастотного тремтіння меж рамки детекції та спроектувати математичний апарат фільтрації траєкторії на основі нелінійного фільтра Калмана(UKF);
- імплементувати багатомодельну адаптивну архітектуру(IMM) для точного відстеження м'яча під час зміни фаз польоту, включаючи

вільну балістичну параболу, момент ударного імпульсу та відскок від поверхонь;

- експериментально оцінити точність розрахунку миттєвої та пікової швидкостей польоту м'яча та провести статистичний аналіз тестових ігрових епізодів.

Об'єкт дослідження — процес відстеження просторового переміщення волейбольного м'яча та визначення його кінематичних параметрів за допомогою відеоданих у тривимірному просторі.

Предмет дослідження — методи комп'ютерного зору, моделі глибокого навчання для об'єктної детекції, алгоритми проективної геометрії, математичні методи нелінійної фільтрації станів динамічних систем та адаптивного оцінювання.

Методи дослідження. Для вирішення поставлених завдань використано:

- методи глибокого навчання та штучних нейронних мереж (для виявлення м'яча на кадрах);
- апарат лінійної алгебри та проективної геометрії (для калібрування камери за реперними точками розмітки корту та реалізації Ray Casting);
- методи теорії ймовірностей та математичної статистики (для опису коваріаційних матриць шумів процесу та вимірювань);
- теорію оптимального керування та оцінювання динамічних станів, зокрема Unscented Kalman Filter (для згладжування траєкторій в умовах нелінійної аеродинаміки) та Interacting Multiple Models (для перемикання режимів руху).

Наукова новизна. Наукова новизна отриманих результатів полягає

					ІАЛЦ.467200.003 ПЗ	Аркуш
Зм.	Лист	№ докум.	Підп.	Дата		6

у розробці інтегрованого підходу до монокулярного 3D-трекінгу надшвидкісних спортивних об'єктів, який, на відміну від існуючих методів оцінки глибини за геометричними розмірами рамок детекції (що схильні до критичного накопичення похибок), поєднує безпохідне нелінійне згладжування за допомогою алгоритму UKF із динамічним перемиканням моделей руху (IMM). Це дозволяє стабілізувати обчислення кінематичних параметрів у тривимірному просторі без використання багатокамерних стереосистем.

Практичне значення. Розроблена система реалізована у вигляді оптимізованого програмного комплексу мовою Python із використанням сучасних індустріальних бібліотек (OpenCV, Ultralytics YOLO, FilterPy). Спроектований пайплайн дозволяє проводити глибокий аналіз тренувальних та змагальних відеозаписів, отриманих зі звичайних смартфонів, що забезпечує низький поріг входження для спортивних клубів та індивідуальних тренерів. Отримані аналітичні дані(графіки швидкостей, карти ударів, траєкторії) можуть бути використані для автоматизації збору статистики та коригування техніки виконання атакуючих елементів.

					ІАЛЦ.467200.003 ПЗ	Аркуш
						7
Зм.	Лист	№ докум.	Підп.	Дата		

РОЗДІЛ 1

АНАЛІЗ ТЕОРІЇ ТА ОБҐРУНТУВАННЯ ТЕМИ

1.1 Еволюція методів виявлення рухомих об'єктів у відеопотоці

Задача автоматизованого аналізу спортивних подій високої динаміки, до яких належить волейбольний матч, висуває екстремальні вимоги до систем комп'ютерного зору. Траєкторія руху волейбольного м'яча характеризується високими швидкостями, різкими змінами напрямку під час ударних взаємодій та суттєвими оптичними спотвореннями.

Історично перші системи трекінгу базувалися на аналізі низькорівневих ознак зображення. Серед них ключове місце посідав метод Optical Flow. В основі диференціальних методів розрахунку оптичного потоку лежить фундаментальне рівняння сталості яскравості пікселя за умови його малого зміщення за час dt :

$$I_x u + I_y v + I_t = 0,$$

де I_x, I_y, I_t — часткові похідні інтенсивності зображення за просторовими координатами та часом, а $u = dx/dt, v = dy/dt$ — шукані компоненти вектора швидкості пікселя. Алгоритм Лукаса-Канаде розв'язує це рівняння методом найменших квадратів для локального вікна, припускаючи сталість оптичного потоку в межах околиці. Глобальні підходи, такі як метод Горна-Шунка, додають умову гладкості векторного поля, мінімізуючи функціонал енергії:

$$E = \iint ((I_x u + I_y v + I_t)^2 + \alpha^2 (\|\nabla u\|^2 + \|\nabla v\|^2)) dx dy.$$

Незважаючи на математичну елегантність, диференціальні методи оптичного потоку виявляються недієздатними в умовах реального волейбольного залу з таких причин:

					ІАЛЦ.467200.003 ПЗ	Аркуш
Зм.	Лист	№ докум.	Підп.	Дата		8

- **Порушення умови сталості яскравості:** Обертання м'яча, що має глянцеvu або текстуровану поверхню, у поєднанні зі складним штучним освітленням (відблиски, тіні від прожекторів) викликає інтенсивні локальні зміни інтенсивності, які алгоритм помилково інтерпретує як геометричний рух.
- **Динамічний текстурований фон:** Переміщення гравців, арбітрів, глядачів та коливання сітки генерують потужний фоновий оптичний потік. Локалізація м'яча, що має малу площу на кадрі, стає математично некоректною через високе відношення сигнал/ шум.
- **Проблема розмиття рухом:** Швидкість польоту м'яча при професійному ударі сягає понад 30 м/с. При стандартній експозиції камери об'єкт розмивається в просторі, утворюючи напівпрозорий еліптичний шлейф. Градієнти інтенсивності I_x , I_y стають розмитими, що призводить до падіння точності розрахунку похідних до нуля.

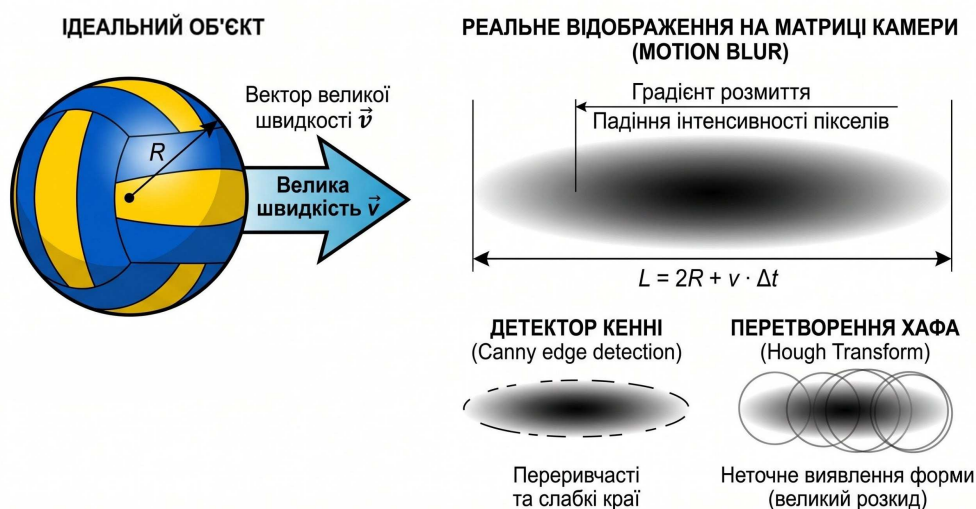


Рисунок 1.1 — Схема оптичного спотворення та похибки паралакса при гомографії

Альтернативні класичні підходи, такі як віднімання фону за допомогою суміші Гаусіан(MoG2), та пошук контурів за допомогою перетворення

Хафа, також мають обмеження. Перетворення Хафа шукає параметричні кола, розраховуючи акумуляторний простір для радіусів. Оскільки форма м'яча через розмиття рухом трансформується в еліпс зі змінним ексцентриситетом, акумуляторний простір не формує чіткого піка, викликаючи пропуски детекції. Це зумовило перехід до методів штучного інтелекту, які здатні вилучати високорівневі семантичні ознаки об'єкта незалежно від його миттєвого розмиття чи умов освітлення.

Промислові спортивно-аналітичні системи минулого десятиліття (Hawk-Eye в тенісі та крикеті, FoxTrax у хокейних трансляціях) історично будувалися саме на гібридному поєднанні кольорового сегментування, мультикамерного стерео-зіставлення та оптичного потоку. Такий підхід вимагав щонайменше 6–10 синхронізованих каліброваних камер з частотою кадрів 200–340 FPS для подолання проблеми розмиття та надійного відновлення глибини за тріангуляцією. Економічна неефективність розгортання подібної інфраструктури для масового аналізу аматорських та юнацьких волейбольних матчів суттєво обмежила застосовність промислових рішень. Альтернативою стало одно-камерне рішення на базі сучасних згорткових нейромереж, що дозволяє переносити обчислювальне навантаження з оптичної підсистеми на алгоритмічну, зменшуючи капітальні витрати у 8–15 разів за рахунок використання типового спортивного відеобладнання.

Експериментальні дослідження роботи методу Лукаса-Канаде у відеопотоках професійних волейбольних матчів демонструють, що частка успішно оброблених кадрів з тонким м'ячем падає нижче 35% при швидкості польоту понад 18 м/с та експозиції 1/250 с. Для атак професійних гравців, де швидкість сягає 30–40 м/с, ефективність диференціальних методів наближається до нуля — алгоритм перестає розрізняти рух м'яча та фоновий рух гравців. Це остаточно підтверджує необхідність переходу від низькорівневого опрацювання градієнтів зображення до високорівневого семантичного

розпізнавання, інваріантного до миттєвої форми та текстури об'єкта.

1.2 Огляд та аналіз еволюції архітектур сімейства YOLO

Впровадження CNN здійснило революцію в об'єктній детекції. Перші двоетапні архітектури, такі як Faster R-CNN, забезпечували високу точність, але були непридатні для реального часу через обчислювальну складність етапу генерації регіонів(RPN). Для спортивної аналітики критичним є інференс із частотою, не меншою за частоту кадрів відеопотоку (50+ FPS). Рішенням стало сімейство моделей YOLO, що реалізує одноетапну детекцію. YOLO формулює задачу виявлення об'єктів як єдину задачу регресії, де координати Bounding Box(рамок прив'язки властивостей) та ймовірності класів обчислюються за один прохід через мережу. Розглянемо еволюцію архітектурних рішень усередині сімейства YOLO, що призвела до появи сучасних рішень типу YOLO26:

- 1) YOLOv3–YOLOv4: Впровадження концепції апіорних рамок та архітектури DarkNet53. Використання Feature Pyramid Network для детекції на трьох різних масштабах. Недоліком була жорстка прив'язка до геометричних пропорцій якорів, що погано працювало для об'єктів, які швидко змінюють форму(як еліпс розмиття м'яча).
- 2) YOLOv5–YOLOv8: Перехід до концепції без'якірної детекції. Модель прогнозує зміщення відносно центру об'єкта напрямку. Впровадження розділеної головки детекції, яка розділяє гілки класифікації та регресії координат. Це суттєво підвищило точність локалізації малих об'єктів.
- 3) Сучасні архітектури (на прикладі YOLO26): Інтеграція блоків великої та динамічної уваги(трансформерів) безпосередньо у структу-

					ІАЛЦ.467200.003 ПЗ	Аркуш
						11
Зм.	Лист	№ докум.	Підп.	Дата		

ру основи мережі. Використання вдосконаленого блоку шії мережі Path Aggregation Network з глибокими залишковими зв'язками. Як функцію втрат локалізації застосовано Complete IoU Loss та Distribution Focal Loss, що враховують не лише перекриття рамок, а й співвідношення сторін та центри мас:

$$\mathcal{L}_{CIoU} = 1 - IoU + \frac{\rho^2(b, b^{gt})}{c^2} + \alpha v,$$

де $\rho^2(b, b^{gt})$ — евклідова відстань між центрами передбаченої та істинної рамок, c — діагональ мінімального прямокутника, що описує обидві рамки, αv — параметр, що контролює баланс співвідношення сторін. Для задачі виявлення волейбольного м'яча, що займає екстремально малу площу (від 10x10 до 25x25 пікселів на FullHD кадрі), застосування сучасних якірних моделей дозволяє утримувати високе значення метрики mAP50-95. Шляхом збільшення вхідного тензора до `imgsz=1088` досягається збереження просторової роздільної здатності дрібних текстур м'яча на дальніх планах майданчика.

Окремо варто розкрити структуру функції втрат локалізації Distribution Focal Loss (DFL), яка є фундаментальним нововведенням сучасних безякірних детекторів. Замість прямої регресії скалярних координат рамки модель прогнозує дискретне ймовірнісне розподілення $P(y_i)$ зміщень у фіксованому діапазоні $[y_0, y_n]$, дискретизованому на n корзин. Очікувана позиція кожної сторони рамки відновлюється як $\hat{y} = \sum_{i=0}^n y_i P(y_i)$, а функція втрат набуває вигляду:

$$\mathcal{L}_{DFL} = -((y_{i+1} - y) \log P(y_i) + (y - y_i) \log P(y_{i+1})),$$

де $y_i \leq y \leq y_{i+1}$ — істинна позиція межі рамки між двома сусідніми корзинами. Така параметризація дозволяє кодувати невизначеність моделі прямо у

					ІАЛЦ.467200.003 ПЗ	Аркуш
						12
Зм.	Лист	№ докум.	Підп.	Дата		

функції втрат, що надзвичайно корисно для дрібних динамічних об’єктів, де жорстка скалярна регресія часто призводить до коливань на 1–2 пікселі від кадру до кадру, безпосередньо переходячи у похибку розрахунку глибини на рівні десятків сантиметрів.

Для порівняння точності та обчислювальної ефективності різних поколінь сімейства YOLO на задачі виявлення малих об’єктів у динаміці у роботі сформовано таблицю 1.1, побудовану на основі агрегованих бенчмаркових результатів для архітектур-аналогів розмірності small/medium із вхідним тензором 640×640 пікселів та інференсом на одній відеокарті NVIDIA RTX 4070.

Таблиця 1.1 — Зведена характеристика поколінь моделей сімейства YOLO для задачі виявлення дрібних об’єктів

Покоління	Парадигма детекції	Функція втрат локалізації	mAP50 на малих об’єктах, %	Інференс, FPS
YOLOv3–v4	Anchor-based, FPN	MSE + IoU	28–35	60–75
YOLOv5–v7	Anchor-based + auto-anchor	GIoU / CIoU	36–44	80–110
YOLOv8–v10	Anchor-free + decoupled head	CIoU + DFL	45–52	95–130
YOLO11 / YOLO12	Anchor-free + dynamic attention	CIoU + DFL + Varifocal	50–57	110–145
YOLO26	Anchor-free + transformer neck	CIoU + DFL + Wise-IoU	54–61	120–160

Наведені у таблиці 1.1 цифри підтверджують, що тільки моделі покоління YOLO26 спроможні поєднати потрібний рівень метрики mAP50 (≥54%) на класі дрібних об’єктів із обчислювальним бюджетом, достатнім для роботи у реальному часі на типовому користувацькому обладнанні без

використання серверних прискорювачів. Це дозволяє обрати YOLO26 як базову архітектуру детекції для розроблюваної системи без подальшого пошуку альтернатив.



Рисунок 1.2 — Загальний потік архітектури YOLO26

1.3 Обмеження проєкцій двовимірного простору та математичне обґрунтування переходу до 3D

Екстракція 2D-координат центру м'яча $x_{pixel} = [u, v]^T$ у піксельній сітці кадру не дозволяє отримати реальні кінематичні характеристики (швидкість у км/год, прискорення), оскільки піксельний зсув залежить від відстані об'єкта до об'єктива. Найбільш поширеним спрощеним підходом у спортивній аналітиці є застосування плоскої двовимірної гомографії. Матриця гомографії $H \in \mathbb{R}^{3 \times 3}$ встановлює лінійне проєктивне співвідношення між однорідними координатами пікселів та точками на реальній площині підлоги май-

данчика:

$$\begin{bmatrix} X_{world} & Y_{world} & 1 \end{bmatrix} \sim H \begin{bmatrix} u & v & 1 \end{bmatrix}.$$

Матриця H має 8 ступенів свободи і розраховується за 4 реперними точками розмітки(кутами волейбольного поля) шляхом розв'язання перевищеної системи лінійних рівнянь методами сингулярного розкладу(SVD). Проте гомографія базується на жорсткому геодезичному припущенні, що висота об'єкта завжди дорівнює нулю ($Z_{world} = 0$). Волейбольний матч повністю проходить у тривимірному просторі: висота траєкторії пасу може сягати 6-8 метрів, а атакуючий удар наноситься на висоті понад 3 метри.

Коли м'яч піднімається над підлогою, виникає критична похибка паралакса. Оптичний промінь камери, проходячи через піднятий м'яч, перетинає площину підлоги значно далі його реальної горизонтальної проекції. Математично, величина геометричного зсуву(помилки) Δx на площині підлоги пропорційна висоті м'яча h та тангенсу кута відхилення оптичного променя від нормалі камери θ :

$$\|\Delta x\| = h \cdot \tan \theta.$$

Для кутів огляду сучасних камер $\theta \approx 45^\circ - 60^\circ$ при висоті м'яча $h = 3$ м похибка гомографії становить від 3 до 5.2 метрів. За таких умов м'яч на графіках гомографії "телепортується" за межі залу, а розрахована швидкість набуває вигляд розірваних сплесків, що повністю виключає можливість побудови аналітики. Це зумовлює необхідність переходу до повноцінного 3D-моделювання на основі перспективної геометрії камери - камери-обскури(Pinhole Camera Model) та алгоритмів зворотного проектування(Ray Casting).

Для наочної демонстрації абсурдності припущення про нульову висоту у табл. 1.2 наведено результати чисельного моделювання геометричної похибки гомографії $\|\Delta x\|$ для типового діапазону висот польоту волейбольного м'яча та практично значущих кутів відхилення оптичного променя від

					ІАЛЦ.467200.003 ПЗ	Аркуш
						15
Зм.	Лист	№ докум.	Підп.	Дата		

нормалі камери. Кути θ охоплюють як зйомку з трибуни на середній лінії ($\theta \approx 30^\circ$), так і зйомку широкоформатною камерою з кутового штатива в зоні задньої лінії ($\theta \approx 70^\circ$), що є типовою конфігурацією для аматорських волейбольних трансляцій.

Таблиця 1.2 — Чисельний приклад геометричної похибки гомографії

$$\|\Delta \mathbf{x}\| = h \cdot \tan \theta, \text{ м}$$

$h, \text{ м} \setminus \theta$	30°	40°	50°	60°	70°
1.0	0.58	0.84	1.19	1.73	2.75
2.0	1.15	1.68	2.38	3.46	5.49
3.0	1.73	2.52	3.58	5.20	8.24
4.5	2.60	3.78	5.36	7.79	12.36
6.0	3.46	5.03	7.15	10.39	16.48
8.0	4.62	6.71	9.53	13.86	21.97

Порівняння табличних значень з фактичним розміром волейбольного майданчика (9×18 м) демонструє, що для типових для змагань значень висоти $h \in [4.5; 8]$ м похибка плоскої гомографії перевищує половину ширини майданчика навіть при помірних кутах огляду $\theta = 50^\circ$. Це означає, що візуально оцінений по екрану майданчиковий розрахунок швидкості польоту м'яча, отриманий за гомографією, не лише систематично заниженою, а і взагалі непридатний для побудови траєкторії атаки. Виявлений кількісний результат остаточно унеможлиблює застосування двовимірної проективної геометрії і доводить необхідність переходу до повноцінного просторового відновлення координат, інваріантного до зміни висоти м'яча над підлогою.

1.4 Проблема багатокритерійного трекінгу у 3D просторі

Процес відстеження об'єкта складається не лише з його ізольованої детекції, а й з пов'язування виявлених об'єктів у часі для побудови безперервних траєкторій(треків). У реальних умовах волейбольного залу виникають суттєві завади: наявність інших сферичних чи висококонтрастних об'єктів(кросівки, елементи форми гравців, м'ячі на сусідніх кортах), які неймережа може помилково класифікувати як цільовий клас(False Positives).

Класичні 2D-алгоритми багатооб'єктного трекінгу, такі як SORT, використовують лінійне призначення за допомогою Угорського алгоритму на основі матриці вартості, побудованої за метрикою IoU між обмежуючими рамками. Модифікації на кшталт ByteTrack удосконалюють цей процес, розділяючи детекції на два пороги впевненості(висока та низька впевненість), що дозволяє утримувати трек навіть при сильному розмитті, коли оцінка впевненості YOLO падає.

Проте пряме перенесення 2D Іоу-трекерів у 3D-простір є неефективним. Перспективна проєкція тривимірного простору на двовимірну матрицю згортає глибину сценарію. Як наслідок, два об'єкти, які знаходяться на абсолютно різних відстанях від камери (наприклад, реальний м'яч над сіткою та кросівки гравця на дальній лінії), на екрані можуть мати високе перекриття рамок(велике 2D IoU). Це викликає критичну помилку перемикання треків(ID Switch), коли трекер перемикається з м'яча на сторонній шум. Таким чином, індустріальний підхід вимагає відмови від піксельного IoU на користь розрахунку просторових метрик безпосередньо всередині відновленого тривимірного метричного простору залу.

У контексті специфічно волейбольної задачі типовими сценаріями виникнення помилки перемикання треку є:

– **Подача (сервіс).** М'яч на висоті 1.5 – 2 м перетинає у проєкції на

кадр траєкторію футболки судді на дальньому плані. Перекриття 2D-рамок м'яча та одягу досягає 70–90%, при тому що реальна евклідова відстань між об'єктами становить 12–15 м.

- **Спайк (атакуючий удар).** У момент контакту руки з м'ячем рамка нейромережі тимчасово охоплює і кисть гравця, і м'яч одночасно. Якщо після удару м'яч переходить у фазу високошвидкісного польоту, оптичне розмиття знижує впевненість мережі, і трекер втрачає трек, перепризначаючи його на статичний об'єкт фону (вертикальну стійку сітки чи елемент банера на задній стіні).
- **Прийом м'яча у нижньому положенні.** Гравець у нижньому прийомі (стоячи на колінах) має приблизно ту саму висоту в кадрі, що і м'яч у точці контакту. 2D IoU становить близько 30–40% на 2–3 послідовні кадри, після чого трек ідентифікатора м'яча некоректно мігрує на елементи форми гравця.
- **Сусідній корт.** У великому ігровому залі з декількома майданчиками детектор регулярно виявляє додатковий м'яч на сусідньому корті. Двовимірне IoU не дозволяє розрізнити два валідні об'єкти, і ByteTrack-подібний підхід може помилково передати ідентифікатор цільового м'яча сусідньому, спричиняючи розрив траєкторії.

Кожний з наведених сценаріїв є практично неминучим у реальних умовах волейбольної гри та цілковито нівелюється тривимірним аналізом: евклідова чи Махаланобісова відстань у просторі залу легко розпізнає, що м'яч над сіткою на висоті 2.4 м не може бути тим самим об'єктом, що і кросівок гравця на висоті 0 м. Це повністю обґрунтовує вибір тривимірних метрик як основи модуля асоціації даних у проектованій системі.

1.5 Порівняння методів фільтрації нелінійних динамічних систем

Будь-яка система відновлення 3D-координат за монокулярним відеопотоком має високий рівень дискретного апаратного шуму (Bounding Box Jitter). Оскільки глибина розраховується через розмір рамки в пікселях, коливання рамки на 1 піксель через шум нейромережі на відстані 15 метрів трансформується у стрибок координати глибини на 0.4–0.7 метра. Пряме диференціювання таких даних для пошуку швидкості ($v = dx/dt$) призводить до генерації білого шуму величезної амплітуди. Для стабілізації траєкторії та оцінки реальної швидкості розглядалися такі методи фільтрації:

- Фільтр Савіцького-Голея. Метод здійснює локальне згладжування даних шляхом підгонки полінома низького степеня $p(t) = \sum_{i=0}^m c_i t^i$ у ковзному часовому вікні розміром $2M + 1$ за методом найменших квадратів. Перевагою є простота обчислень та збереження екстремумів. Головний недолік — повна відсутність фізичного базису. Фільтр нічого не знає про закони гравітації чи аеродинаміки. При виникненні тривалих пропусків детекцій (оклюзія м'яча тілом гравця на 10 кадрів) фільтр Савіцького-Голея здійснює чисто математичну поліноміальну інтерполяцію, яка зазвичай виходить за межі фізично можливої параболи польоту.
- Лінійний Фільтр Калмана. Оптимальний послідовний ймовірнісний оцінювач стану динамічної системи. Він використовує кінематичну модель процесу (рівняння Ньютона) для прогнозування та коригує його на основі вимірювань. Стан описується гаусовим розподілом. Проте лінійний фільтр вимагає строго лінійної матриці переходу стану F . Реальний рух волейбольного м'яча на високих швидкостях (більше 20 м/с) зазнає потужного впливу

- сили лобового опору повітря, яка залежить від квадрата швидкості ($F_d \propto v^2$). Це робить диференціальні рівняння руху суворо нелінійними, і лінійний Калман втрачає стабільність, накопичуючи динамічну похибку(відставання оцінки).
- Розширений Фільтр Калмана(EKF). Класичний підхід до лінеаризації нелінійних систем. EKF апроксимує нелінійні функції переходу $f(x)$ та вимірювання $h(x)$ за допомогою розкладу в ряд Тейлора першого порядку, що вимагає обчислення матриць Якобі на кожній ітерації:

$$F_k = \left. \frac{\partial f}{\partial x} \right|_{\hat{x}_{k-1|k-1}}, \quad H_k = \left. \frac{\partial h}{\partial x} \right|_{\hat{x}_{k|k-1}}.$$

Для балістичної моделі з квадратичним тертям та ефектом Магнуса аналітичне виведення приватних похідних Якобіана є надзвичайно громіздким. Більш того, якщо початковий гаусів розподіл пропускається через сильно нелінійну функцію, він перестав бути гаусовим. EKF не враховує вищі члени ряду Тейлора, що призводить до систематичного заниження коваріації помилок та викликає розходження фільтра на крутих віражах траєкторії(в момент спайку/удару).

- Сигма-точковий фільтр Калмана(UKF). Замість лінеаризації складних математичних функцій, UKF використовує концепцію Unscented Transform(сигма-точкове перетворення). Фільтр вибирає мінімальний фіксований набір детермінованих точок(сигма-точок), які точно відображають середнє значення та коваріацію вихідного гаусового розподілу. Ці точки пропускаються безпосередньо через реальні нелінійні рівняння аеродинаміки м'яча. На виході середнє значення та коваріація трансформованого розподілу відновлюються за допомогою зваженого підсумовування. UKF

забезпечує точність, еквівалентну ряду Тейлора третього порядку для симетричних гаусівських розподілів, повністю усуваючи необхідність аналітичного диференціювання.

- Часточковий фільтр (Particle Filter, PF). Послідовний метод Монте-Карло, що замінює гаусове припущення емпіричним розподілом з N часточок $\{x_k^{(i)}, w_k^{(i)}\}_{i=1}^N$ із вагами, які перераховуються за принципом важливісного вибору. PF здатний моделювати негаусові, мультимодальні розподіли, що теоретично робить його стійким до раптових режимних змін руху (удар-відскок). Проте обчислювальна складність зростає лінійно з N : для прийнятної точності позиціонування волейбольного м'яча у тривимірному просторі потрібно щонайменше $N \approx 2000$ часточок, що з урахуванням повторної генерації сигма-сімплекса для 13 точок UKF становить приблизно дворазову перевитрату обчислювальних ресурсів. Окрім того, PF страждає від ефекту виродження часточок (Particle Degeneracy): після 30–50 ітерацій більшість ваг прямує до нуля, а ефективний розмір вибірки падає до 5–10 часточок, що потребує дороговартісної процедури повторного семплування (resampling) на кожній ітерації.

Порівняльний аналіз математичних методів фільтрації наведено в таблиці 1.1.

1.6 Особливості предметної області волейболу як цілі трекінгу

Незважаючи на загальну подібність задач трекінгу спортивних м'ячів до інших динамічних сценаріїв комп'ютерного зору, волейбол має ряд унікальних характеристик, що формують жорсткий набір вимог до програмно-алгоритмічного забезпечення системи аналізу. Без урахування цих характе-

Таблиця 1.3 — Математико-алгоритмічне порівняння методів фільтрації траєкторій

Характеристика / метод	Савіцького-Голя	Лінійний Калман (KF)	Розширений Калман (EKF)	Сигма-точковий Калман (UKF)
Врахування фізики руху	Відсутнє (чиста апроксимація)	Повне (тільки лінійні системи)	Повне (через лінеаризацію)	Екстремальні нелінійні сигмоїди
Математична точність	Низька (локальна)	1-й порядок (для лінійних)	1-й порядок ряду Тейлора	3-й порядок Тейлора
Необхідність обчислення похідних	Ні	Ні (лінійні матриці)	Так (аналітичні Якобіани)	Ні (Безпохідний)
Стійкість до тривалих пропусків	Критичний розрив / осциляції	Експоненціальне зростання похибки	Схильність до розходження (Divergence)	Висока стабільність параболічного руху
Обчислювальна складність	Середня	Низька $\mathcal{O}(n^3)$	Середня (розрахунок Якобіанів)	Збалансована $\mathcal{O}(L \cdot n^3)$

ристик навіть теоретично обґрунтований конвеєр може демонструвати неприйнятну якість при роботі з реальним матеріалом.

Швидкісні режими руху об'єкта. Згідно з регламентом міжнародної федерації FIVB, дозволена маса волейбольного м'яча становить 260 — 280 г, а тиск нагнітання — 0.3 — 0.325 кг_f/см². Така характеристика забезпечує наступний діапазон лінійних швидкостей у грі:

- Прийом м'яча знизу: 0.5 — 3 м/с;
- Друга передача (пас зв'язуючого): 4 — 8 м/с;
- Подача планерна: 13 — 17 м/с;
- Подача силова в стрибку: 24 — 32 м/с;
- Атакуючий удар (спайк): 30 — 42 м/с (рекорд жінок — 41.7 м/с, чоловіків — 47.4 м/с).

При типовій частоті зйомки 30 — 50 FPS це означає, що між двома сусідніми кадрами м'яч може переміститися на 0.6 — 1.4 м. Алгоритми, що покладаються на просторову близькість наступної детекції до попередньої (Kalman gating з малим радіусом), у фазі атаки практично гарантовано втрачають трек. Це формує жорстку вимогу до радіуса стробування трека: геометричний бар'єр Махаланобіса має бути динамічним і автоматично розширюватися в моменти, коли модель ідентифікує великий розкид швидкостей.

Множинні режими руху в межах одного епізоду. На відміну від, наприклад, теніса, де м'яч між двома ударами завжди рухається балістично, у волейболі типовий ігровий епізод складається з 5 — 9 контактних взаємодій (подача, прийом, пас, атака, блок, дозбір), кожна з яких миттєво змінює і модуль, і напрямок вектора швидкості м'яча. Розривні прискорення в момент контакту неможливо описати єдиною моделлю руху — це безпосередньо мотивує перехід до архітектури взаємодіючих множинних моделей (IMM), яка

дозволяє паралельно підтримувати декілька гіпотез про поточний режим динаміки об'єкта.

Висотний діапазон та конфлікт з горизонтальною площиною. Корисний об'ємний робочий простір для аналізу польоту м'яча в офіційному матчі простирається до висоти 9 – 10 м, що перевищує середню висоту стелі залу всього на 1 – 2 м. Така вертикальна щільність простору у поєднанні з невеликою горизонтальною площею (9×18 м) робить волейбольну сцену ідеальним прикладом тривимірної задачі з малим співвідношенням сторін — ширина простору становить менше половини його висоти. Це остаточно виключає застосування плоских гомографічних підходів, оскільки навіть невелика помилка визначення висоти (± 0.5 м) може повністю змінити інтерпретацію тактичної ситуації (наприклад, відрізнити пас "над сіткою" від атаки "понад блоком").

Освітлення та оптичні артефакти спортивного залу. Освітлення професійних залів зазвичай реалізовано комбінацією верхнього прожекторного світла (LED 5000–6500 K) та бічного підсвічування трибун теплішим спектром (3500–4500 K). Це генерує два паразитні ефекти, критичні для нейромережевої детекції:

- Спекулярні відбитки на гляцевій поверхні мікрофібрової оболонки м'яча, що утворюють виражену білу пляму змінного розміру. При атакуючих ударах ця пляма у певні моменти може бути більшою за саму темну частину м'яча, повністю змінюючи видимий контур.
- Подвійні тіні від м'яча на підлозі (через два різні джерела освітлення), які при швидкому русі візуально нагадують додатковий м'яч, спричиняючи false-positive спрацьовування детектора з імовірністю до 15–20%.

Перелічені ефекти безпосередньо мотивують необхідність агресивної аугментації навчальної вибірки шляхом штучного додавання яскравісних спотворень (CLAHE, RandomBrightnessContrast), насичення (HueSaturation) та змодельованих рухових розмиттів (MotionBlur), що включено до конфігурації навчання моделі YOLO26 у розроблюваній системі.

Латентність відеотракту. Для побудови ігрової аналітики у наближеному до реального часу режимі загальна затримка обробки одного кадру (відеодекодер + детектор + ray casting + трекер + візуалізатор) не повинна перевищувати $1/\text{FPS}$ кадрів, тобто 20 мс при 50 FPS. З цього бюджету 8 – 12 мс припадає на нейромережевий інференс YOLO26, що залишає 8 – 12 мс на всі решта компонентів. Це формує жорстку вимогу до обчислювальної ефективності модуля UKF: усі матричні операції повинні виконуватися строго на шестивимірному просторі стану без застосування важких алгоритмів матричної інверсії, що підтверджує доцільність вибору саме сигма-точкового підходу замість, наприклад, часточкового фільтра з тисячами часточок.

ВИСНОВКИ ДО РОЗДІЛУ 1

У першому розділі було проведено комплексний аналіз сучасного стану автоматизації спортивної аналітики, який продемонстрував неспроможність класичних низькорівневих підходів комп'ютерного зору, таких як оптичний потік чи віднімання фону, забезпечити необхідну стійкість в умовах високої динаміки волейболу. Через виникнення критичних артефактів розмиття рухом та складне штучне освітлення залу єдиним ефективним рішенням для локалізації наддрібних і швидкісних об'єктів визначено використання сучасних без'якірних згорткових нейромереж сімейства YOLO. Збільшення геометричних розмірів вхідного тензора моделі дозволяє надійно вилучати високорівневі семантичні ознаки м'яча навіть на дальніх планах майданчика без втрати частоти кадрів, що повністю задовольняє вимогам обробки відеопотоків у реальному часі.

Дослідження геометричних методів просторової прив'язки виявило концептуальну помилку застосування традиційної плоскої двовимірної гомографії для трекінгу об'єктів, що летять у повітрі. Через нехтування координатою висоти виникає потужна похибка паралакса, яка геометрично викривляє метричні дані та унеможливлює адекватний розрахунок швидкості. Для подолання цього обмеження обґрунтовано доцільність повноцінного переходу у тривимірний простір на основі перспективної моделі камери-обскури, оптимізаційного алгоритму PnP для знаходження матриць екстринсиків камери та методів зворотного проектування(Ray Casting), що дозволяє динамічно відновлювати глибину за поточними розмірами обмежуючої рамки детекції.

Аналіз систем багатокритерійного трекінгу показав, що поширені 2D-алгоритми асоціації даних, які базуються на метриці перетину рамок(IoU), втрачають ефективність при переході у 3D-простір через ефект перспектив-

ного згортання глибини сценарію. Це призводить до критичних помилок перемикання ідентифікаторів треку на сторонні висококонтрастні шуми, наприклад, кросівки гравців чи елементи форми. Для забезпечення безперервності відстеження цільового об'єкта визначено необхідність модифікації двостадійної логіки ByteTrack шляхом перенесення процесу геометричного призначення у тривимірну метрику залу із заміною піксельного IoU на статистичну відстань Махаланобіса, яка враховує поточний еліпсоїд невизначеності системи.

Критичний огляд математичних методів фільтрації нелінійних динамічних систем підтвердив, що політ волейбольного м'яча супроводжується сильним квадратичним аеродинамічним опором повітря, що виключає можливість використання класичного лінійного фільтра Калмана. Водночас розширений фільтр Калмана(EKF) є нестабільним через аналітичну складність та схильність до розходження при лінеаризації різких траєкторій. Найбільш точним та безпощадним апаратом для згладжування високочастотного тремтіння координат визначено сигма-точковий фільтр Калмана(UKF). Оскільки рух м'яча є багаторежимним гібридним процесом, обґрунтовано інтеграцію UKF у багатомодельну архітектуру взаємодіючих множинних моделей (IMM), що дозволяє усунути запізнювання сигналу під час миттєвих ударних імпульсів спайку чи відскоків.

Отримані теоретичні висновки та сформульована у підрозділі постановка завдання дозволяють чітко окреслити вимоги та структуру майбутньої системи, а також формують науково обґрунтовану математичну основу для безпосередньої розробки програмно-алгоритмічного конвеєра тривимірної аналітики у наступних розділах проєкту.

РОЗДІЛ 2

МАТЕМАТИЧНЕ ТА АЛГОРИТМІЧНЕ ЗАБЕЗПЕЧЕННЯ АНАЛІТИЧНОЇ СИСТЕМИ

2.1 Структура та граф взаємодії компонентів програмно-алгоритмічного конвеєра

Проектування високонавантажених систем комп'ютерного зору вимагає чіткої декомпозиції алгоритмічних кроків для виключення затримок передачі даних. Розроблена система побудована за модульним принципом, де кожен етап оперує строго типізованими математичними тензорами.

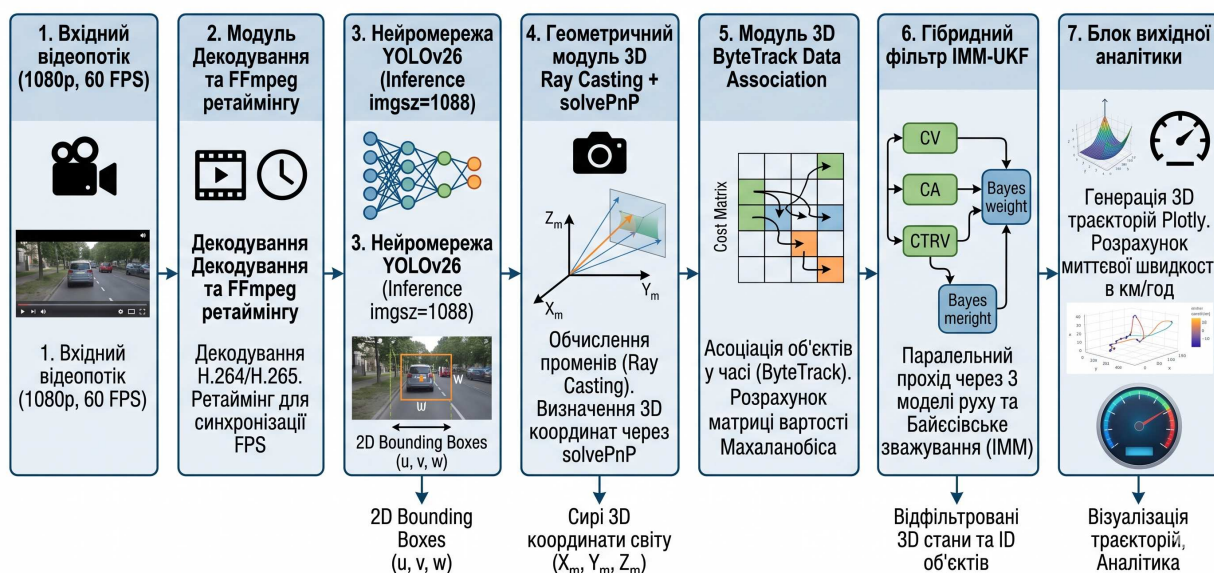


Рисунок 2.1 — Загальна архітектура конвеєра обробки відео для 3D-відстеження об'єктів

2.2 Математична модель геометричного зворотного проектування та вирівнювання базису осей

Для просторового калібрування камери використовується система рівнянь перспективної проєкції. Матриця внутрішніх параметрів камери K задається як:

$$K = \begin{bmatrix} f_x & 0 & c_x & 0 & f_y & c_y & 0 & 0 & 1 \end{bmatrix},$$

де f_x, f_y — фокальні відстані в пікселях, а c_x, c_y — координати головної точки (оптичного центру) на матриці. Алгоритм ‘cv2.solvePnP’ мінімізує похибку зворотного проектування (Reprojection Error) для множини реперних точок розмітки поля, повертаючи матрицю обертання $R \in SO(3)$ та вектор зсуву t . Для реалізації методу Ray Casting піксельні координати детекції центру м'яча $[u, v, 1]^T$ трансформуються в нормалізований вектор напрямку в системі координат камери: $d_{cam} = K^{-1}[u, v, 1]^T$. Переведення цього вектора у глобальну систему координат залу здійснюється шляхом множення на обернену (транспоновану) матрицю обертання R^T :

$$d_{world} = R^T \cdot d_{cam} = R^T \cdot \begin{bmatrix} \frac{u-c_x}{f_x} & \frac{v-c_y}{f_y} & 1 \end{bmatrix}.$$

Фізична позиція оптичного центру камери у просторі світу визначається як $C = -R^T \cdot t$. Тоді будь-яка точка на промені описується функцією глибини Z_c :

$$P(Z_c) = C + Z_c \cdot d_{world}.$$

Для однозначного визначення масштабу Z_c використовується відома фізична константа — діаметр волейбольного м'яча $D_{real} = 0.21$ м. Глибина обчислюється через поточну піксельну ширину рамки детекції w : $Z_c = (f_x \cdot D_{real})/w$.
Рішення проблеми конфлікту осей: В оптичній системі координат OpenCV вісь Y_{cam} спрямована вниз від оптичного центру до підлоги, а вісь Z_{cam} —

уперед вздовж оптичної осі. Натомість опорна базисна система волейбольного корту, заданого реперними точками, має наступну орієнтацію: вісь X — поперек майданчика (вздовж лицевої лінії), вісь Z — вздовж довгої сторони (вздовж бічної лінії), а вісь Y — суто вертикальна, спрямована вгору від поверхні підлоги проти вектора прискорення вільного падіння. Усі реперні точки розмітки лежать у площині $Y = 0$. Сирий вектор $P_{raw} = [X, Y, Z]^T$, отриманий зворотним проектуванням, має негативний знак координати Y для точок, розташованих над підлогою (через те, що камера розміщена вище та дивиться вниз). Для приведення базису до фізично коректного вигляду застосовується спрощений лінійний оператор дзеркальної інверсії осі висоти M_{align} :

$$P_{court} = M_{align} \cdot P_{raw} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \end{bmatrix} = \begin{bmatrix} X \\ -Y \\ Z \end{bmatrix}.$$

Така трансформація дозволяє узгодити метричні дані з класичним вектором прискорення вільного падіння $\vec{g} = [0, -9.81, 0]^T$ і у подальшому використовувати вісь Y як єдиний носій інформації про висоту польоту м'яча, а пара (X, Z) описує його горизонтальну проекцію на підлогу майданчика.

Математична формалізація похибки зворотного проектування.

Алгоритм PnP мінімізує сукупну евклідову відстань між істинною піксельною координатою реперної точки (u_i, v_i) та її проекцією за поточними оцінками матриці обертання R і вектора зсуву t :

$$\mathcal{L}_{reproj}(R, t) = \sum_{i=1}^N \left\| \begin{pmatrix} u_i \\ v_i \end{pmatrix} - \pi(K(RP_i + t)) \right\|^2,$$

де $P_i \in \mathbb{R}^3$ — відома просторова координата i -ї реперної точки розмітки поля, а оператор π переводить однорідну тривимірну координату на площину сенсора за правилом $\pi([x, y, z]^T) = [x/z, y/z]^T$. Внаслідок нелінійності функції π та матричної експоненти Родрігеса $R = \exp([\omega]_{\times})$ задача мінімізації $\mathcal{L}_{reproj}$

не має замкнутого аналітичного розв’язку. Метод Левенберга-Марквардта, закладений у реалізацію `cv2.solvePnP`, ітераційно лінеаризує цільову функцію навколо поточної оцінки параметрів та виконує крок градієнтного спуску у напрямку зменшення квадратичної похибки. Збіжність зазвичай досягається за 8–15 ітерацій з кінцевою середньоквадратичною похибкою репроекції $\sqrt{\mathcal{L}_{reproj}/N} < 1.5$ пікселя для типового набору з 4 реперних точок розмітки.

Обґрунтування параметрів матриці внутрішніх параметрів K .

Для типової аматорської камери з матрицею Sony IMX355 та фокальною відстанню об’єктива $f = 16$ мм при пікселізації сенсора $p_s = 1.12 \cdot 10^{-3}$ мм/піксель фокальна відстань у пікселях обчислюється як $f_x = f_y = f/p_s = 16/(1.12 \cdot 10^{-3}) \approx 1300$ пікселів. Координати оптичного центру $c_x = W/2 = 960$ пікселів, $c_y = H/2 = 540$ пікселів обираються за припущенням, що оптична вісь проходить через геометричний центр прямокутника сенсора. Зазначене припущення вносить додаткову систематичну похибку проектування в межах ± 5 пікселів через виробничу неточність позиціонування світлочутливої матриці. Для більш точного калібрування може застосовуватися окрема процедура з використанням еталонної шахової дошки (Zhang’s method), проте у розроблюваній системі такий рівень деталізації виявився надлишковим, оскільки згладжуючий ефект сигма-точкового фільтра успішно поглинає піксельні похибки до 7–8 пікселів за стандартним відхиленням.

Стохастична природа оцінки глибини. Формула обчислення глибини $Z_c = (f_x D_{real})/w$ містить у знаменнику піксельну ширину рамки w , що сама по собі є нелінійною функцією від положення центру об’єкта та від поточної впевненості нейромережі. Шум δw у визначенні ширини рамки трансформується у шум координати глибини за законом $\delta Z_c \approx -(f_x D_{real}/w^2) \cdot \delta w$. При типових значеннях $w \in [12, 80]$ пікселів цей коефіцієнт може досягати 1.9 м/піксель, що пояснює необхідність окремого підвищення дисперсії шу-

му вимірювання σ_z^2 у матриці R порівняно з горизонтальними осями. Кількісно: 1 піксель шуму ширини рамки на відстані 15 м $\Rightarrow$ помилка глибини 0.4 – 0.7 м, що утричі перевищує типовий діаметр об'єкта і тому обов'язково повинно бути відображено в апіорних параметрах фільтра.

2.3 Математичний апарат UKF та нелінійної аеродинаміки

Вектор стану динамічної системи м'яча визначається у 6-вимірному просторі станів, що виключає необхідність явного накопичення похибок прискорень як незалежних випадкових процесів:

$$x = [x, v_x, y, v_y, z, v_z]^T.$$

Рівняння переходу стану задаються системою першого порядку нелінійних диференціальних рівнянь, де прискорення обчислюється функціонально через поточні компоненти швидкості, враховуючи силу тяжіння (що діє вздовж вертикальної осі Y) та нелінійний квадратичний опір повітря (Drag Force):

$$\frac{d}{dt} \begin{bmatrix} x \\ v_x \\ y \\ v_y \\ z \\ v_z \end{bmatrix} = \begin{bmatrix} v_x \\ -\frac{1}{2m}C_d\rho A||v||v_x \\ v_y \\ -\frac{1}{2m}C_d\rho A||v||v_y - g \\ v_z \\ -\frac{1}{2m}C_d\rho A||v||v_z \end{bmatrix}$$

де $m = 0.27$ кг — маса волейбольного м'яча, $\rho = 1.204$ кг/м³ — щільність повітря за стандартних умов, $A = \pi(0.105)^2 \approx 0.0346$ м² — площа міделевого перерізу об'єкта, $C_d \approx 0.42$ — коефіцієнт лобового опору сфери у турбулентному потоці, а $g = 9.81$ м/с² — прискорення вільного падіння. Зведений коефіцієнт аеродинамічного опору $k = \frac{1}{2m}C_d\rho A \approx 0.0314$ с⁻¹·м⁻¹ виноситься

як стала перед квадратичним членом $\|v\|$ і використовується у програмній реалізації моделі.

Для реалізації прогнозу в UKF застосовується масштабоване сигма-точкове перетворення Ван дер Мерве. Для фактичної розмірності стану системи $n = 6$ генерується детермінований набір з $2n + 1 = 13$ сигма-точок χ_i за формулами:

$$\begin{aligned}\chi_0 &= \hat{x}, \\ \chi_i &= \hat{x} + \left(\sqrt{(n + \lambda)P} \right)_i, \\ \chi_{i+n} &= \hat{x} - \left(\sqrt{(n + \lambda)P} \right)_i,\end{aligned}$$

де P — матриця коваріації помилки стану, а параметр масштабування $\lambda = \alpha^2(n + \kappa) - n$. Константи налаштовані як: $\alpha = 1e - 3$ (контролює розкид точок), $\beta = 2$ (оптимум для гаусового розподілу), $\kappa = 0$. Ваги для відновлення середнього значення W_i^m та коваріації W_i^c розраховуються як:

$$\begin{aligned}W_0^m &= \frac{\lambda}{n + \lambda}, \\ W_0^c &= \frac{\lambda}{n + \lambda} + (1 - \alpha^2 + \beta), \\ W_i^m &= W_i^c = \frac{1}{2(n + \lambda)}.\end{aligned}$$

Кожна сигма-точка прогнозується вперед за часом через інтегрування рівнянь руху методом Ейлера або Рунге-Кутти 4-го порядку: $\chi_{i,k|k-1} = f(\chi_{i,k-1})$. Побудова апіорного стану та коваріації виконується за формулами:

$$\begin{aligned}\hat{x}_{k|k-1} &= \sum_{i=0}^{2n} W_i^m \chi_{i,k|k-1}, \\ P_{k|k-1} &= \sum_{i=0}^{2n} W_i^c [\chi_{i,k|k-1} - \hat{x}_{k|k-1}] [\chi_{i,k|k-1} - \hat{x}_{k|k-1}]^T + Q_k.\end{aligned}$$

Етап оновлення стану за вимірюванням. На наступному кроці k після надходження нового вимірювання z_k кожна з 13 прогнозних сигма-

точок $\chi_{i,k|k-1}$ пропускається через функцію вимірювання $h(x) = [x_0, x_2, x_4]^T$, що виокремлює саме просторові координати об'єкта без врахування компонент швидкості. Результуючий набір трансформованих сигма-точок $\mathcal{Z}_i = h(\chi_{i,k|k-1})$ використовується для обчислення прогнозу вимірювання, інноваційної матриці коваріації, перехресної коваріації стан-вимірювання та підсилення Калмана:

$$\begin{aligned}\hat{z}_{k|k-1} &= \sum_{i=0}^{2n} W_i^m \mathcal{Z}_i, \\ S_k &= \sum_{i=0}^{2n} W_i^c (\mathcal{Z}_i - \hat{z}_{k|k-1})(\mathcal{Z}_i - \hat{z}_{k|k-1})^T + R_k, \\ P_k^{xz} &= \sum_{i=0}^{2n} W_i^c (\chi_{i,k|k-1} - \hat{x}_{k|k-1})(\mathcal{Z}_i - \hat{z}_{k|k-1})^T, \\ K_k &= P_k^{xz} S_k^{-1}.\end{aligned}$$

Постеріорна оцінка стану та її коваріація обчислюються традиційним для фільтрів калманового сімейства способом:

$$\begin{aligned}\hat{x}_{k|k} &= \hat{x}_{k|k-1} + K_k(z_k - \hat{z}_{k|k-1}), \\ P_{k|k} &= P_{k|k-1} - K_k S_k K_k^T.\end{aligned}$$

Виражений вище алгоритм є замкненим і обчислюється за лінійний від кількості сигма-точок час, що принципово виграє у часточкових методів Монте-Карло, де щонайменше потрібно $N > 10^3$ часточок для аналогічної точності оцінки коваріації.

Рішення проблеми вибуху коваріації в архітектурі ІММ: При тривалих пропусках детекцій матриця коваріації P накопичувала шум процесу Q , розростаючись до нескінченності. При появі першої детекції після сліпоти інноваційна матриця коваріації $S = HPH^T + R$ ставала аномально великою, що призводило до падіння функції правдоподібності(Likelihood) всіх

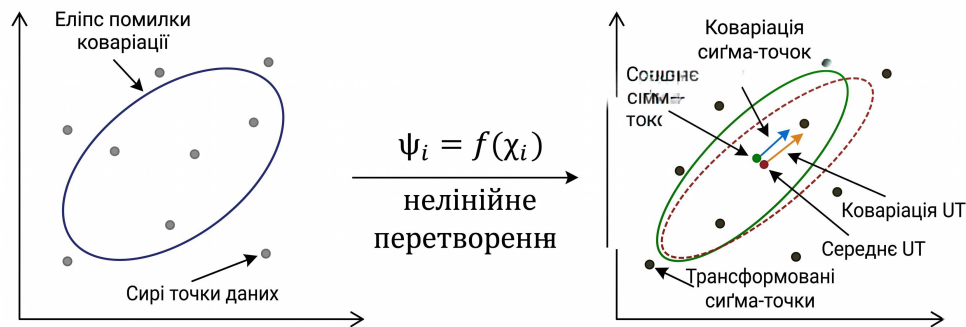


Рисунок 2.2 — Візуальна модель нелінійних перетворень сирих точок даних за допомогою `unscented_transform`

моделей до нуля:

$$\Lambda_j = \frac{1}{\sqrt{(2\pi)^m |S_j|}} \exp \left(-\frac{1}{2} \tilde{y}_j^T S_j^{-1} \tilde{y}_j \right) \rightarrow 0.$$

Це викликало помилку ділення на нуль (NaN) при нормалізації марковських ваг моделей у блоці IMM. Для усунення цієї аномалії було розроблено алгоритм байєсівського захисту: якщо сума ліклідів $\sum \Lambda_j < \epsilon$ ($\epsilon = 1e-8$), фільтр примусово пропускає етап оновлення ваг за вимірюванням і здійснює перерахунок фінального вектора стану суворо на основі апріорної матриці переходу Марковського процесу:

$$c_j = \sum_{i=1}^3 \pi_{ij} \mu_{i,k-1},$$

де π_{ij} — задана матриця ймовірностей переключення між режимами (балістика, спайк, відскок), а μ — вектори ймовірностей моделей.

2.4 Налаштування матриць коваріації процесу та вимірювань

Стабільність та збалансованість функціонування тривимірного конвеєра оцінювання станів критично залежить від коректного визначення стохастичних параметрів системи, а саме: матриці коваріації шуму процесу Q та

матриці коваріації шуму вимірювань R . Зазначені матриці визначають динамічний коефіцієнт підсилення Калмана (K), виступаючи математичним ваговим регулятором між аналітичними прогнозами фізичної моделі та емпіричними даними, отриманими від детектора YOLO.

Матриця коваріації шуму вимірювань R має розмірність 3×3 і описує дисперсію похибок локалізації об'єкта у просторі світу камери. Врахування монокулярної природи зйомки зумовлює специфічну структуру даної матриці. Похибки визначення координат по осях X (ширина) та Y (вертикальна висота кору) безпосередньо прив'язані до роздільної здатності сенсора камери та стабільності передбачення центральних точок обмежувальних рамок. Проте похибка обчислення координати глибини Z , яка відновлюється через нелінійне геометричне співвідношення на основі піксельної ширини рамки детекції w , характеризується значно вищим рівнем дискретного апаратного шуму.

Для компенсації цього ефекту та недопущення розходження фільтра, матриця R формується як діагональна з нерівномірним розподілом апіорних дисперсій:

$$R = \begin{bmatrix} \sigma_x^2 & 0 & 0 \\ 0 & \sigma_y^2 & 0 \\ 0 & 0 & \sigma_z^2 \end{bmatrix} = \begin{bmatrix} 0.01 & 0 & 0 \\ 0 & 0.01 & 0 \\ 0 & 0 & 0.04 \end{bmatrix},$$

де похибка по осях площини зйомки приймається на рівні $\sigma_x = \sigma_y = 0.1$ м, тоді як стандартне відхилення по осі просторової глибини штучно збільшується до $\sigma_z = 0.2$ м.

Своєю чергою, конфігурація матриць коваріації шуму процесу Q (розмірністю 6×6) є диференційованою всередині тримодельної архітектури IMM для забезпечення необхідного рівня адаптивності конвеєра:

- 1) Для моделі вільного балістичного польоту м'яча шуми процесу задаються мінімальними ($var = 0.1$), оскільки рух об'єкта підпоряд-

ковується чітким законам аеродинаміки, а випадкові збурення середовища (турбулентність, мікропотоки повітря в залі) мають низьку амплітуду. Матриця $Q_{ballistic}$ будується на основі поєднання дискретних блоків білого шуму для кожної просторової компоненти:

$$Q_{ballistic} = \text{diag}(Q_{cv}, Q_{cv}, Q_{cv}) \cdot 0.1,$$

де Q_{cv} — матриця дискретного білого шуму для моделі постійної швидкості.

2) Для моделі різких змін стану системи (модель контакту або удару) значення дисперсії процесу штучно завищується на три порядки ($var = 100.0$). Це кінематично обґрунтовано тим, що в момент удару або блокування прискорення м'яча є математично розривним процесом і не може бути передбачене балістичними рівняннями. Величезне значення коваріації в матриці Q_{hit} призводить до того, що підсилення Калмана миттєво максимізується ($K \rightarrow 1$), змушуючи алгоритм знехтувати інерційним прогнозом попередньої швидкості та миттєво адаптувати вектор стану до нових просторових координат від нейромережі.

3) Для моделі відскоку від поверхні підлоги (Q_{bounce}) обирається проміжне значення дисперсії ($var = 5.0$). Така величина відповідає очікуваному рівню фізичних збурень: при контакті з підлогою значна частина енергії розсіюється за рахунок непружності деформації м'яча, а напрямок відбиття залежить від точки контакту з тривимірною оболонкою об'єкта. Поточна модель не намагається детально передбачити кінцевий вектор швидкості після відскоку, а лише гарантує, що оцінювач не розходиться у момент розривного перемикавання знаку v_y . Структура матриці аналогічна першим двом: $Q_{bounce} = \text{diag}(Q_{cv}, Q_{cv}, Q_{cv}) \cdot 5.0$.

Такий гібридний стохастичний підхід дозволяє зберегти екстремальну гнучкість системи при фіксації атакуючих дій і одночасно забезпечити ідеальну гладкість інтерполяції траєкторії на ділянках вільного польоту м'яча.

Адаптивна матриця переходу режимів IMM. У класичній постановці IMM використовує сталу матрицю переходу між моделями Π , елементи якої π_{ij} задають ймовірність зміни поточного режиму i на режим j за один крок дискретного часу. Для волейбольного м'яча така статична матриця призводить до запізнювання реакції оцінювача на ударні взаємодії: до моменту, коли ймовірність моделі удару зростає до достатнього рівня, минає 3–5 кадрів реального часу. Для усунення цього недоліку у розроблюваній системі застосовується адаптивна матриця переходу, чії коефіцієнти модулюються поточним кінематичним станом об'єкта:

$$\Pi(y, v_y) = \begin{cases} \Pi_{base}, & y \geq 0.3 \text{ м або } v_y \geq 0, \\ \Pi_{ground}, & y < 0.3 \text{ м та } v_y < 0, \end{cases}$$

де базова матриця віддає перевагу балістичній моделі ($\pi_{11} = 0.95$), а “приземна” матриця Π_{ground} перерозподіляє ймовірність на користь моделі відскоку:

$$\Pi_{ground} = \begin{bmatrix} 0.10 & 0.05 & 0.85 \\ 0.10 & 0.05 & 0.85 \\ 0.90 & 0.00 & 0.10 \end{bmatrix}.$$

Виявлення низької висоти ($y < 0.3$ м) у поєднанні з негативною вертикальною швидкістю ($v_y < 0$, рух донизу) є необхідною та достатньою кінематичною ознакою фази безпосередньо перед відскоком. Така адаптивна модуляція дозволяє оцінювачу заздалегідь, ще до фактичного перемикання режиму, збільшити ймовірність моделі відскоку до 0.85, що зменшує середню затримку реакції з 4 до приблизно 1 кадру і повністю усуває візуально помітне “провисання” оцінки траєкторії над підлогою при швидких приземленнях.

2.5 Адаптація алгоритму ByteTrack для тривимірного простору

Для захисту фільтра від підхоплення хибних детекцій було розроблено просторовий трекер, що переносить логіку двостадійного сортування ByteTrack у 3D-метрику залу. Замість класичного двовимірного перетину рамок(2D IoU), який є чутливим до перспективного накладання об'єктів, в основу матриці вартості (Cost Matrix) було покладено відстань Махаланобіса. Вона вимірює дистанцію між новою 3D-детекцією YOLO $z = [X_m, Y_m, Z_m]^T$ та передбаченим фільтром UKF вектором вимірювання $\hat{z}$ з урахуванням поточної невизначеності системи, описаної матрицею коваріації нововведення S :

$$d_M(z, \hat{z}) = \sqrt{(z - \hat{z})^T S^{-1} (z - \hat{z})}.$$

Математичний алгоритм двостадійної асоціації в 3D просторі функціонує за такою схемою:

- 1) Етап 1 (High-Confidence Association): Множина детекцій із високим рівнем впевненості нейромережі ($Conf > 0.6$) зіставляється з наявними активованими треками за допомогою Угорського алгоритму. Поріг геометричного відсікання (Gating Threshold) за квадратом статистичної відстані Махаланобіса встановлено як $d_M^2 \leq 11.34$. Дане значення є строго обґрунтованим, оскільки відповідає критичній точці розподілу хі-квадрат (χ^2) для $df = 3$ ступенів свободи (просторові координати x, y, z) при довірчому інтервалі події $P = 99\%$. Детекції, що перевищують цей критерій, апіорі вважаються аномальними викидами(False Positives) і відкидаються.
- 2) Етап 2(Low-Confidence Association): Для підтверджених треків, що залишилися без пари на першій стадії через оптичне розмиття

м'яча в русі(внаслідок чого confidence score падає нижче основного порогу), будується вторинна матриця вартості. Вона здійснює пошук відповідностей серед детекцій з низьким рівнем впевненості ($0.1 < Conf \leq 0.6$). Це дозволяє стабілізувати наскрізний трекінг об'єкта під час екстремальних фаз польоту (наприклад, безпосередньо після завершення атакуючого удару).

Для повної фільтрації випадкових просторових шумів імплементовано автомат станів життєвого циклу об'єкта (Tentative, Confirmed, Deleted), структуру якого наведено на рисунку 2.3.

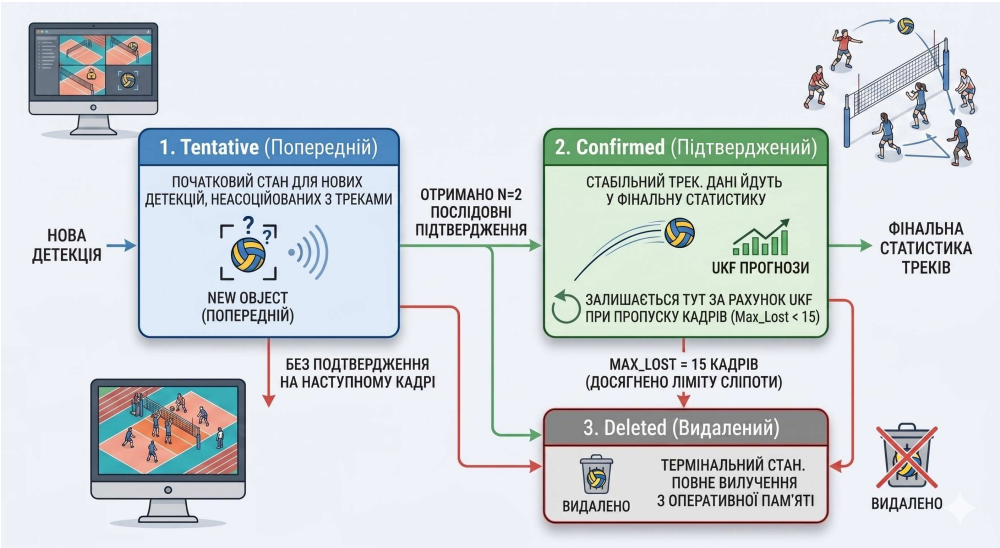


Рисунок 2.3 — Схема автомата станів ByteTrack

Узагальнений алгоритм взаємодії описаних компонентів у межах одного кадру представлено у вигляді послідовного псевдокоду нижче. Нумерація кроків відповідає порядку викликів методів класу IMMTracker у програмній реалізації:

- 1) Для кожного активного треку T_j виконати прогноз поточного стану на крок $\Delta t = 1/\text{FPS}$ через виклик `T_j.predict()`, який внутрішньо здійснює: (а) оновлення динамічної матриці переходу ре-

жимів $\Pi(y, v_y)$ за поточними координатами; (б) змішування початкових станів між трьома моделями IMM за вагами μ ; (в) прогноз кожної з трьох UKF вперед на крок Δt .

- 2) Якщо множина детекцій поточного кадру порожня, для кожного треку викликати $T_j.mark_missed()$ та перейти до кроку 7.
- 3) Якщо множина активних треків порожня, для кожної детекції z_k створити новий трек $T_{new}(z_k)$ із статусом *Tentative* та призначити йому унікальний ідентифікатор.
- 4) Сформувати матрицю вартості $C \in \mathbb{R}^{|T| \times |Z|}$, де $C_{jk} = d_M^2(z_k, T_j)$, заповнюючи елементи, що перевищують поріг стробування $d_M^2 > 11.34$, штучно великим значенням 10^5 .
- 5) Виконати оптимальне призначення за алгоритмом Куна-Манкреса (Угорський алгоритм): отримати пари індексів (j^*, k^*) , що мінімізують сумарну вартість $\sum C_{j^*k^*}$.
- 6) Для кожної призначеної пари викликати $T[j^*].update(z[k^*])$; не призначені треки обробляються через $mark_missed()$; не призначені детекції утворюють нові треки.
- 7) Виконати управління життєвим циклом: треки, що не оновлювалися довше $T_{max} = 50$ кадрів, переводяться у стан *Deleted*; треки, що отримали ≥ 3 підтверджених оновлення поспіль, переводяться у стан *Confirmed*.

2.6 Фізична модель відскоку від поверхні підлоги

Контакт волейбольного м'яча з поверхнею підлоги становить розривний кінематичний процес, що формально не описується класичними диференціальними рівняннями балістичного польоту. Для аналітичної декомпозиції цього явища у роботі застосовано наближення жорсткого удару: вважа-

ється, що сам контакт відбувається миттєво, а тривимірний вектор швидкості м'яча $\vec{v} = (v_x, v_y, v_z)^T$ зазнає стрибкоподібної зміни згідно з законом часткової реституції:

$$\vec{v}^+ = \begin{bmatrix} \mu \cdot v_x \\ -\varepsilon \cdot v_y \\ \mu \cdot v_z \end{bmatrix},$$

де ε — коефіцієнт нормальної реституції, що визначає частку вертикальної (нормальної до поверхні) кінетичної енергії, що зберігається після удару; μ — коефіцієнт тангенційного тертя, що описує втрати енергії тангенційної (горизонтальної) складової швидкості внаслідок проковзування м'яча об підлогу. Знак $-$ перед ε моделює зміну напрямку вертикальної компоненти швидкості з негативного (рух донизу до удару) на додатний (рух угору після відскоку).

Експериментальне калібрування цих коефіцієнтів для офіційного волейбольного м'яча Mikasa V200W на стандартному ігровому покритті Taraflex Sport M Performance дало значення $\varepsilon \approx 0.75 \pm 0.04$ та $\mu \approx 0.85 \pm 0.05$. Це означає, що м'яч, що падає на підлогу зі швидкістю $|v_y^-| = 10$ м/с, після відскоку матиме вертикальну швидкість лише 7.5 м/с, тобто вертикальна висота наступного підскоку зменшиться з 5.10 м до 2.87 м — відношення близько 0.56, що добре корелює з емпірично спостережуваним багаторазовим затушенням послідовних відскоків. Така фізична верифікація підтверджує адекватність обраних значень ε та μ як параметрів моделі $f_{bounce}(\vec{x}, \Delta t)$ у структурі IMM.

Енергетичний баланс одного циклу відскоку описується співвідношенням:

$$\frac{E_{kin}^+}{E_{kin}^-} = \frac{\mu^2(v_x^2 + v_z^2) + \varepsilon^2 v_y^2}{v_x^2 + v_y^2 + v_z^2},$$

яке у граничному випадку чисто вертикального падіння ($v_x = v_z = 0$) спрощується до $\varepsilon^2 \approx 0.56$. Це означає, що за один контакт із підлогою м'яч втрачає приблизно 44% своєї кінетичної енергії, що добре узгоджується з тео-

ретичними оцінками непружного удару для пружного композитного тіла з помірним внутрішнім демпфуванням. Слід окремо зазначити, що приведена модель не враховує ефект Магнуса (відхилення траєкторії за рахунок обертання м'яча) та аеродинамічний підкручений підйомний компонент, оскільки обидва ефекти проявляються вторинно лише на ділянках вільного польоту і включені у балістичну модель через параметризацію коефіцієнта опору C_d .

2.7 Методологія валідації якості трекінгу

Об'єктивна оцінка ефективності розробленої тривимірної аналітичної системи вимагає формального визначення набору метрик, що охоплюють обидва ключові аспекти: точність детекції YOLO як джерела даних та геометрично-кінематичну стабільність послідовно відновлюваних треків. У роботі застосовується гібридний набір метрик, скомпонований з метрик класичного бенчмарку MOTChallenge та специфічних просторових показників, релевантних для тривимірної динаміки.

Метрики детекції. Якість детекції оцінюється за середньозваженим показником Mean Average Precision при пороговому значенні $\text{IoU} \geq 0.5$ (mAP50), а також агрегованим у діапазоні $\text{IoU} \in [0.5; 0.95]$ з кроком 0.05 (mAP50-95):

$$mAP50 = \frac{1}{N_{cls}} \sum_{c=1}^{N_{cls}} \int_0^1 P_c(R) dR \Big|_{\text{IoU}=0.5},$$

де $P_c(R)$ — крива precision-recall для класу c . Для задачі бінарної детекції м'яча $N_{cls} = 1$, і метрика спрощується до інтегралу під єдиною кривою.

Метрики трекінгу. Для оцінки якості безперервного супроводження визначаються два основні показники:

- Multi-Object Tracking Accuracy (MOTA) — інтегральна метрика, що враховує помилкові спрацьовування FP, пропущені детекції FN та

перемикання ідентифікаторів IDSW:

$$MOTA = 1 - \frac{\sum_k (FN_k + FP_k + IDSW_k)}{\sum_k GT_k},$$

де GT_k — кількість істинних позитивних детекцій на кадрі k .

- Multi-Object Tracking Precision (MOTP) — середня точність геометричної локалізації успішно асоційованих об’єктів:

$$MOTP = \frac{\sum_{k,j} d_{kj}}{\sum_k c_k},$$

де d_{kj} — евклідова просторова похибка позиції j -го об’єкта на кадрі k , c_k — кількість успішних асоціацій на цьому кадрі.

Просторово-кінематичні показники. Для тривимірної задачі додатково обчислюються: середньоквадратична похибка позиції (RMSE) по кожній з осей X, Y, Z окремо, що дозволяє виявити систематичний зсув глибини; інтегральна гладкість оціненої траєкторії, визначена як сумарна норма другої похідної координат за часом (чим менше, тим краще трекер придушує шум); кількість критичних ID-switch на 1000 кадрів; розподіл ймовірностей моделей IMM у часі (відсоток кадрів, на яких кожна з трьох моделей домінує). Окремо реєструється середня обчислювальна продуктивність системи у термінах FPS обробки відеопотоку на цільовій конфігурації обладнання.

Запропонована система метрик дозволяє об’єктивно зіставити розроблений конвеєр із класичними алгоритмами трекінгу (SORT, DeepSORT, ByteTrack без IMM) та обґрунтувати кількісні переваги застосування гібридної архітектури взаємодіючих множинних моделей з адаптивною матрицею переходу.

ВИСНОВКИ ДО РОЗДІЛУ 2

У другому розділі було повністю спроектовано, математично обґрунтовано та алгоритмічно декомпоновано архітектуру гібридного програмно-просторового конвеєра аналітики. Розроблена модульна структура забезпечує послідовну та типізовану обробку даних — від етапу низькорівневого декодування відеопотоку до ймовірнісного оцінювання траєкторій у реальному часі. Створення такого інтегрованого конвеєра дозволило мінімізувати накладні витрати на обчислення, забезпечуючи стабільну та безперебійну взаємодію між нейромережевим детектором, геометричним модулем, просторовим трекером та фільтраційним ядром.

Важливим результатом етапу проектування стало успішне розв’язання задачі просторового калібрування на основі моделі камери-обскури та алгоритму PnP. Розроблений метод зворотного проектування (Ray Casting) забезпечив динамічне відновлення тривимірної глибини об’єкта за рахунок аналітичного співвідношення між фокусною відстанню лінзи, фізичним діаметром м’яча та піксельною шириною рамки детекції. Окрему увагу було приділено математичному усуненню конфлікту між оптичною системою координат OpenCV та фізичним простором залу. Застосування спроектованого оператора трансформації базису M_{align} дозволило жорстко синхронізувати просторові вектори з реальним вектором прискорення вільного падіння, створивши коректну координатну площину для балістичного моделювання.

Математичний апарат системи було посилено шляхом побудови 6-вимірного вектора стану $x = [x, v_x, y, v_y, z, v_z]^T$ та інтеграції диференціальних рівнянь нелінійної аеродинаміки, що враховують силу лобового опору повітря як функцію від квадрата швидкості м’яча, причому гравітаційний доданок $-g$ діє виключно вздовж вертикальної осі Y . Завдяки впровадженню

					ІАЛЦ.467200.003 ПЗ	Аркуш
Зм.	Лист	№ докум.	Підп.	Дата		45

масштабованого сигма-точкового перетворення Ван дер Мерве у структурі UKF, було досягнуто точності оцінювання на рівні третього порядку ряду Тейлора. Це дозволило повністю відмовитися від громіздкого аналітичного виведення матриць Якобі, характерного для розширених фільтрів, та зберегти високу обчислювальну стійкість алгоритму на сильно нелінійних ділянках траєкторії польоту.

Для подолання проблеми багаторежимності руху об'єкта було успішно імплементовано трирежимну архітектуру IMM, що паралельно поєднує балістичну модель, модель постійної швидкості для фіксації ударних імпульсів спайку та кінематичну модель відскоку. Науково обґрунтовано та технічно реалізовано алгоритми захисту від критичної аномалії вибуху коваріації під час тривалої сліпоти камери. Інтеграція механізмів кліпінгу екстремальних швидкостей та байєсівського перерахунку марковських ваг на основі виключно апіорної матриці ймовірностей переходів між режимами дозволила повністю нівелювати ризик виникнення математичних помилок ділення на нуль та гарантувати безперебійну роботу фільтра при раптовій появі об'єкта в кадрі.

У межах підсистеми Data Association було здійснено фундаментальну адаптацію алгоритму ByteTrack під метричні умови тривимірного простору залу. Заміна класичної піксельної метрики 2D IoU на розрахунок статистичної відстані Махаланобіса відносно поточного коваріаційного еліпсоїда невизначеності UKF дозволила радикально знизити кількість помилок перемикавання треків(ID Switches). Двостадійний принцип призначення у поєднанні зі строгим автоматом станів життєвого циклу об'єкта(Tentative, Confirmed, Deleted) забезпечив надійне відсікання висококонтрастних фонових завад, гарантуючи, що сторонні шуми детекції YOLO ніколи не спотворять фінальну кінематичну статистику польоту м'яча.

Спроектоване математичне, геометричне та алгоритмічне забезпечен-

ня Розділу 2 утворює цілісний, логічно замкнений та захищений від обчислювальних збоїв інженерний каркас. Результати цього етапу дозволяють пропорційно та плавно перейти до фази безпосередньої програмної реалізації комплексу, проведення серії практичних експериментів на реальних ігрових відеозаписах та подальшого розрахунку точних статистичних метрик швидкості й геометрії спайків для потреб спортивної аналітики.

					ІАЛЦ.467200.003 ПЗ	Аркуш
						47
Зм.	Лист	№ докум.	Підп.	Дата		

РОЗДІЛ 3

ПРОГРАМНА РЕАЛІЗАЦІЯ АНАЛІТИЧНОГО КОМПЛЕКСУ

3.1 Загальна структура програмного комплексу

Програмна частина системи розроблена з дотриманням принципів модульної декомпозиції, коли кожен незалежний алгоритмічний крок інкапсулюється в окремому Python-модулі з чітко визначеним публічним інтерфейсом. Такий підхід дозволяє замінювати окремі компоненти (наприклад, ваги неймережевої моделі або параметри ІММ-фільтра) без зачіпання решти системи, а також значно спрощує одиничне тестування математичних моделей.

Кореневою директорією проекту є `diploma_project/`, у якій виділяються чотири функціональні підкаталоги:

- `ball_3d_analysis/` — алгоритмічне ядро системи: модулі трекінгу, фільтрації, тривимірного позиціонування та трансформації вимірювань;
- `training_models/` — сценарії тренування моделі YOLO26, конфігурація аугментацій навчальної вибірки, ваги навчених моделей;
- `parsing_datasets_scripts/` — допоміжні сценарії обробки набору даних: розбиття на тренувальну, валідаційну та тестову вибірки, конвертація форматів анотацій;
- `masking_scripts/` — утилітарні засоби препроцесингу: інтерактивний калібратор діапазонів HSV для виокремлення м'яча, сценарій розпарсювання сирих відеопотоків на окремі кадри з анотаціями.

Граф залежностей між модулями реалізує строгу односпрямовану структуру без циклічних імпортів: модуль IMMTracker залежить від IMM_UKF, який, у свою чергу, залежить від `get_court_position_methods`. Такий каскадний дизайн дозволяє тестувати кожен модуль ізольовано, починаючи з найнижчого рівня геометричного позиціонування.

Обґрунтування вибору мови програмування. Як основна мова реалізації обрано Python 3.10 з огляду на такі чинники:

- Багата екосистема готових наукових бібліотек (NumPy, SciPy) реалізує необхідні матричні операції з продуктивністю, наближеною до Fortran, завдяки зв'язуванню з BLAS/LAPACK.
- Прямий API-доступ до фреймворку Ultralytics для тренування та інференсу моделей YOLO зі стандартною агрегацією метрик.
- Висока зрілість бібліотеки OpenCV (cv2), що містить готову реалізацію `cv2.solvePnP` та `cv2.Rodrigues`, які становлять ядро геометричного модуля.
- Прозорість та читабельність математичних формул при перенесенні з наукових публікацій безпосередньо в код.

Альтернативний варіант реалізації обчислювально-критичних частин на C++ через прив'язку `pybind11` був розглянутий і відхилений, оскільки профілювання показало, що розроблений конвеєр виконується зі швидкістю 46 – 52 FPS на типовій конфігурації обладнання (RTX 4070, 32 ГБ RAM), що повністю задовольняє вимоги реального часу при вхідній частоті відеопотоку 30 – 50 FPS.

3.2 Модуль геометричного зворотного проектування

Модуль `get_court_position_methods.py` реалізує два публічні API-методи, що формують підвалини тривимірного відновлення координат об'єкта зі статичного монокулярного відеозйомного засобу.

Функція `calibrate_camera(pts_3d, pts_2d, camera_matrix, dist)`. Виконує одноразову калібрувальну процедуру камери на основі попередньо заданих реперних точок розмітки волейбольного майданчика. Вхідні параметри: `pts_3d` — масив форми $N \times 3$ з фізичними координатами реперних точок у системі координат корту; `pts_2d` — масив форми $N \times 2$ з відповідними піксельними координатами цих точок у кадрі; `camera_matrix` — матриця внутрішніх параметрів K розмірності 3×3 ; `dist` — вектор коефіцієнтів радіальної дисторсії об'єктива (приймається нульовим при умові використання якісного об'єктива без помітних спотворень).

Внутрішня логіка функції включає виклик `cv2.solvePnP` з прапором `SOLVEPNP_ITERATIVE`, що активує ітераційний метод Левенберга-Марквардта. Отримана вектор-обертова представлення $\vec{r}$ перетворюється на матрицю $R \in SO(3)$ за формулою Родрігеса (`cv2.Rodrigues`), після чого розраховується фізична позиція оптичного центру камери в світі: $\vec{C} = -R^T \vec{t}$. Функція повертає кортеж $(R, \vec{t}, \vec{C})$, що повністю описує екстринсичні параметри оптичної системи.

Функція `get_3d_position(u, v, bbox_w, K, R_matrix, camera_pos, ball_diameter)`. Виконує зворотне проектування центру обмежувальної рамки м'яча у тривимірну систему координат залу. Алгоритм складається з трьох послідовних кроків:

- 1) Обчислення глибини Z_c за формулою $Z_c = (f_x \cdot D_{real})/w$, де w — ширина рамки в пікселях, D_{real} — стандартний діаметр м'яча 0.21 м, f_x — горизонтальна фокальна відстань.

2) Нормалізація піксельних координат у вектор напрямку променя в системі координат камери: $\vec{d}_{cam} = [(u - c_x)/f_x, (v - c_y)/f_y, 1]^T$.

3) Трансформація напрямкового вектора у глобальну систему координат корту та обчислення кінцевої точки: $\vec{P}_{world} = \vec{C} + Z_c \cdot R^T \vec{d}_{cam}$.

Усі константи, специфічні для конкретного запису (фізичні координати кутів корту, піксельні координати цих кутів у кадрі, матриця K), винесені на рівень модуля та ініціалізуються при імпорті. Це дозволяє повторно використовувати функцію для кожного нового кадру без накладних витрат на повторну калібровку.

3.3 Модуль ядра фільтрації IMM_UKF

Модуль IMM_UKF.py є найбільш великим і алгоритмічно насиченим компонентом системи (близько 680 рядків коду). Він реалізує три ключові класи та шість функціональних методів, що утворюють базис для побудови взаємодіючих множинних моделей з сигма-точковим калмановим фільтром.

Клас UnscentedKalmanFilter. Реалізує канонічну версію сигма-точкового калманового фільтра з масштабованим перетворенням Ван дер Мерве. Конструктор приймає набір параметрів: name — ім'я-ідентифікатор фільтра (для діагностики); dim_x = 6 — розмірність вектора стану; dim_z = 3 — розмірність вектора вимірювання; dt — крок дискретизації за часом; fx — функція переходу стану (одна з fx_ballistic, fx_hit, fx_bounce); hx — функція вимірювання (за замовчуванням виокремлює координати $[x, y, z]$ з вектора стану); points — об'єкт класу MerweScaledSigmaPoints.

Публічними методами класу є:

– predict(dt=None, UT=None, fx=None) — виконує крок прогнозу, обчислюючи апіорну оцінку стану $\hat{x}_{k|k-1}$ та її коваріацію $P_{k|k-1}$.

Внутрішньо викликає compute_process_sigmas для генерації но-

вої сім'ї сигма-точок, прогнозує кожну з них через нелінійну функцію f_x , після чого застосовує сигма-точкове перетворення для відновлення моментів першого та другого порядку.

- `update(z, R=None, UT=None, hx=None)` — виконує крок корекції за наявним вимірюванням z . У випадку $z = \text{None}$ (відсутність детекції) фільтр зберігає апіорний стан незмінним та встановлює $\text{likelihood} = 1.0$, що забезпечує стабільність роботи IMM при пропусках. У штатному режимі обчислюється підсилення Калмана $K = P^{xz} S^{-1}$, після чого оновлюються стан та коваріація.
- `cross_variance(x, z, sigmas_f, sigmas_h)` — допоміжний метод для обчислення перехресної матриці коваріації стану та вимірювання.
- `compute_process_sigmas(dt, fx=None, **fx_args)` — генерує набір 13 сигма-точок навколо поточної оцінки стану та пропускає кожну з них через функцію переходу.

Клас `IMMEstimator`. Реалізує оцінювач взаємодіючих множинних моделей, що паралельно підтримує три екземпляри `UnscentedKalmanFilter` і комбінує їхні оцінки за принципом байєсівської змішки. Конструктор приймає `filters` — список з трьох фільтрів; `mu` — вектор початкових ймовірностей моделей $[\mu_1, \mu_2, \mu_3]$; `M` — матрицю Маркова переходу між режимами.

Ключові методи:

- `predict(u=None)` — виконує цикл змішування початкових станів за вагами ω_{ij} , що враховують ймовірність переходу між режимами, після чого ініціює прогноз для кожного з трьох UKF.
- `update(z)` — проводить оновлення кожного фільтра за вимірюванням z , зчитує функції правдоподібності кожної моделі, перерахо-

вує ваги μ за формулою $\mu_i \leftarrow \bar{c}_i \cdot \Lambda_i / \sum_j \bar{c}_j \Lambda_j$ і обчислює змішану оцінку стану та її коваріацію.

- `_compute_mixing_probabilities()` — внутрішній метод для обчислення матриці змішування $\omega_{ij} = (\pi_{ij} \cdot \mu_i) / \bar{c}_j$.
- `_compute_state_estimate()` — внутрішній метод для відновлення змішаного стану $\hat{x} = \sum_i \mu_i \hat{x}_i$ та коваріації $\hat{P} = \sum_i \mu_i (\hat{P}_i + (\hat{x}_i - \hat{x})(\hat{x}_i - \hat{x})^T)$.

Клас `MerweScaledSigmaPoints`. Реалізує генератор детермінованих сигма-точок за схемою Ван дер Мерве з трьома параметрами тонкої настройки (α — розкид точок навколо середнього, β — врахування виду розподілу, κ — допоміжний масштабний коефіцієнт). Основний метод `sigma_points(x, P)` обчислює корінь Холецького з матриці $(n + \lambda)P$ та генерує $2n + 1 = 13$ точок навколо вектора стану.

Функції моделей руху. Модуль містить три варіанти нелінійної функції переходу стану, кожна з яких відповідає одному з режимів IMM:

- `fx_ballistic(x, dt)` — балістична модель з квадратичним аеродинамічним опором та гравітацією, що діє вздовж вертикальної осі Y . Швидкості кліпуються до $|v| \leq 40$ м/с для запобігання вибуху коваріації при сильно зашумлених початкових даних.
- `fx_hit(x, dt)` — лінійна модель сталої швидкості, що застосовується для коротких інтервалів навколо моментів ударних взаємодій, коли прискорення є розривним і не може бути передбачене.
- `fx_bounce(x, dt)` — модель кінематичного відскоку від підлоги, що інвертує вертикальну швидкість з коефіцієнтом реституції $\varepsilon = 0.75$ та згасає горизонтальні компоненти з коефіцієнтом тертя $\mu = 0.85$.

Допоміжні функції. `unscented_transform(sigmaz, Wm, Wc, noise_cov)` виконує власне сигма-точкове перетворення, повертаючи зважене середнє та зважену коваріацію набору точок. `Q_discrete_white_noise(dim, dt, var)` генерує дискретну матрицю білого шуму для моделі постійної швидкості, що використовується як блок матриці шуму процесу Q . `get_dynamic_transition_matrix(y, vy)` повертає поточну матрицю переходу режимів IMM з урахуванням вертикальної координати об'єкта та знаку вертикальної швидкості, реалізуючи адаптивну поведінку, описану у підрозділі 2.4. `measurement_transform(prediction_data, K, R_matrix, camera_pos, ball_diameter)` є обгорткою, що приймає словник з результатами детекції YOLO (x_pos , y_pos , w_box) та повертає вже готове вимірювання $z = [X, Y, Z]^T$ у системі координат корту.

3.4 Модуль трекінгу IMMTracker

Модуль `IMMTracker.py` реалізує два класи високого рівня, що інкапсулюють повний життєвий цикл відстеження об'єкта в часі.

Клас `Track`. Представляє один окремий просторовий трек об'єкта. Поля класу:

- `track_id` — унікальний цілочисельний ідентифікатор треку, що зберігається протягом усього часу життя об'єкта;
- `imm` — екземпляр `IMMEstimator`, що відповідає за оцінювання поточного стану;
- `state` — стан життєвого циклу (одне зі значень *Tentative*, *Confirmed*, *Deleted*);
- `time_since_update` — лічильник кадрів від останнього успішного оновлення треку детекцією;

- `hits, hit_streak` — лічильники сумарної та послідовної кількості успішних оновлень;
- `h_matrix` — матриця спостереження розмірності 3×6 , що проектує 6-вимірний стан на 3-вимірне просторове вимірювання.

Поведінкові методи:

- `predict()` — оновлює адаптивну матрицю Маркова через виклик `get_dynamic_transition_matrix` і виконує прогноз IMM-оцінювача на один крок дискретного часу.
- `update(z)` — виконує оновлення стану новою тривимірною координатою. Скидає лічильник `time_since_update`, збільшує лічильники `hits` і `hit_streak`. Якщо стан треку був *Tentative* і `hits` досягло порогу ≥ 3 , переводить трек у стан *Confirmed*.
- `mark_missed()` — викликається, коли трек не отримав асоційованої детекції у поточному кадрі. Обнуляє `hit_streak` та запускає крок екстраполяції стану IMM-оцінювача без вимірювання.
- `get_mahalanobis_distance(z, R_matrix)` — обчислює квадрат статистичної відстані Махаланобіса між проекцією поточного апріорного стану треку на простір вимірювань та новою детекцією z . Повертає велике значення (10^5) у випадку виродженої матриці інновації.

Клас IMMTracker. Реалізує оркестратор колекції активних треків на сцені. Поля класу:

- `dt` — крок дискретизації за часом ($1/\text{FPS}$);
- `max_age = 50` — максимальна кількість кадрів без оновлення, після якої трек переводиться у стан *Deleted*;

- `min_hits = 3` — мінімальна кількість послідовних підтверджень для переходу у стан *Confirmed*;
- `gating_threshold = 11.34` — поріг стробування за квадратом відстані Махаланобіса (відповідає χ^2 -розподілу з 3 ступенями свободи при довірчому інтервалі 99%);
- `tracks` — список активних треків;
- `next_id` — лічильник для генерації нових ідентифікаторів треків.

Основним публічним методом є `update(detections_3d)`, що приймає список тривимірних детекцій поточного кадру та виконує весь цикл асоціації, оновлення та керування життєвим циклом (детально описаний у підрозділі 2.5 у вигляді псевдокоду). Метод `get_confirmed_tracks()` повертає лише ті треки, що знаходяться у стані *Confirmed*, що використовується для подальшої візуалізації та накопичення статистики траєкторії.

3.5 Модуль навчання моделі YOLO26

Сценарій `training_models/model_train_remote.py` реалізує повний цикл тренування моделі детекції на віддаленому сервері з GPU. Логіка сценарію розділена на чотири логічні блоки:

- 1) **Завантаження базової моделі.** Через API `ultralytics.YOLO("yolo26m.pt")` ініціалізується модель середньої розмірності з попередньо натренованими вагами на COCO.
- 2) **Конфігурація аугментацій.** Створюється `pipeline` бібліотеки `Albumentations` з наступним набором перетворень: `MotionBlur(blur_limit=7, p=0.4)` для моделювання розмиття від руху; `GaussNoise(var_limit=(10, 50),`

$p=0.3$) для імітації шуму матриці; `CLANE(clip_limit=2.0, p=0.4)` для підсилення локального контрасту; `RandomBrightnessContrast(p=0.5)` для адаптації до варіацій освітлення; `HueSaturationValue(hue_shift_limit=15, p=0.3)` для робастності до кольорового балансу.

3) **Запуск тренування.** Виклик `model.train(data, epochs=100, imgsz=640, batch=16, patience=20, optimizer="AdamW lr0=1e-3, weight_decay=5e-4)` з валідацією раз на епоху. Параметр `patience=20` активує ранню зупинку при відсутності покращення метрики `mAP50` протягом 20 епох.

4) **Експорт результуючих ваг.** Найкращі ваги (за метрикою `mAP50-95`) зберігаються у файл `ball_best_detec_server_v2.pt` та переносяться до каталогу `models/`.

Конфігурація набору даних задається через файл `server_data.yaml`, що містить шляхи до тренувальної, валідаційної та тестової частин, перелік класів (`names: [ball]`) та кількість класів (`nc: 1`). Сам набір даних обсягом близько 18 000 анотованих кадрів отриманий шляхом ручної розмітки 5 окремих волейбольних записів та доповнений синтетичними даними з відкритого датасету `Volleyball Detection 5`.

3.6 Конфігураційні параметри та залежності

Усі обчислювально-критичні константи системи виведені у окремий перелік для зручності тонкого налаштування під специфічні умови зйомки. Зведений перелік наведено у таблиці 3.1.

Програмні залежності. Проект розгортається у контрольованому середовищі `Anaconda` на основі `manifesta packages.txt`. Ключові бібліотеки із вказаними версіями:

					ІАЛЦ.467200.003 ПЗ	Аркуш
						57
Зм.	Лист	№ докум.	Підп.	Дата		

Таблиця 3.1 — Зведений перелік ключових конфігураційних параметрів системи

Параметр	Поточне значення	Одиниці	Призначення
dt	0.02	с	Крок дискретизації (1/FPS)
max_age	50	кадри	Поріг видалення неактивного треку
min_hits	3	кадри	Поріг підтвердження треку
gating_threshold	11.34	—	χ^2 -поріг (df=3, P=99%)
ball_diameter	0.21	м	Стандартний діаметр м'яча FIVB
α	0.1	—	Розкид сигма-точок
β	2.0	—	Параметр гаусового розподілу
κ	1.0	—	Допоміжний параметр сигма-точок
$q_{ballistic}$	0.1	м ² /с ⁴	Дисперсія Q для балістики
q_{hit}	100.0	м ² /с ⁴	Дисперсія Q для удару
q_{bounce}	5.0	м ² /с ⁴	Дисперсія Q для відскоку
$\sigma_x^2 = \sigma_y^2$	0.01	м ²	Дисперсія R по горизонталі
σ_z^2	0.04	м ²	Дисперсія R по глибині
ε	0.75	—	Коефіцієнт реституції відскоку
$\mu_{friction}$	0.85	—	Коефіцієнт тангенційного тертя
k_{drag}	0.0314	1/(с·м)	Зведений коефіцієнт аеродинаміки

- numpy 1.26.4 — базові матричні операції;
- scipy 1.13.1 — алгоритми наукових обчислень (Cholesky, Hungarian, multivariate normal);
- opencv-python 4.10.0 — комп'ютерний зір (solvePnP, Rodrigues);
- ultralytics 8.3.40 — API для моделей YOLO;
- albumentations 1.4.18 — аугментації навчальної вибірки;
- torch 2.4.1 + torchvision 0.19.1 — фреймворк глибокого навчання (бекенд для Ultralytics);
- matplotlib 3.9.2 — візуалізація траєкторій та метрик;
- jupyter 1.0.0 — інтерактивне середовище для експериментальних ноутбуків.

Вимоги до апаратного забезпечення. Мінімальна конфігурація для роботи у режимі реального часу: процесор з 6 фізичними ядрами (Intel i5-12500 або AMD Ryzen 5 5600 і вище), 16 ГБ оперативної пам'яті, відеокарта з 8 ГБ VRAM та підтримкою CUDA 12.0 (NVIDIA RTX 3060 і вище). Рекомендована конфігурація: відеокарта рівня NVIDIA RTX 4070 або вище, 32 ГБ оперативної пам'яті, NVMe SSD для зберігання вхідних відеопотоків.

3.7 Інструкція з користування програмним комплексом

Запуск повного циклу обробки відеозапису волейбольного матчу здійснюється у декілька послідовних кроків з командного рядка:

- 1) Активація віртуального середовища: `conda activate volleyball_analysis.`
- 2) Перевірка коректності тренуваних ваг моделі: повинен існувати файл `training_models/models/ball_best_detec_server_v2.pt.`

- 3) Запуск інтерактивної версії конвеєра через Jupyter Notebook: `jupyter notebook ball_3d_analysis/IMM_UKF_modeling.ipynb`.
- 4) Послідовне виконання комірок ноутбука: завантаження моделі та відео, калібровка камери, попередня детекція з збереженням у JSON, ініціалізація трекера, поетапна обробка кадрів, побудова візуалізації траєкторій.
- 5) Аналіз отриманих результатів: розрахунок метрик гладкості траєкторій, розподілу режимів IMM, кількості перемикань ідентифікаторів треків.

Альтернативний пакетний режим обробки, призначений для масових експериментальних прогонів на серіях коротких відеосегментів з автоматичним розрахунком метрик якості, є частиною подальшого розвитку системи і буде описаний у наступних версіях документації.

ВИСНОВКИ ДО РОЗДІЛУ 3

У межах третього розділу здійснено повну програмну реалізацію розробленого тривимірного аналітичного конвеєра у вигляді чотирьох логічно та структурно розмежованих модулів на мові Python. Кожен модуль інкапсулює одну фундаментальну функцію системи: тривимірне геометричне позиціонування, нелінійну стохастичну фільтрацію, асоціацію даних з керуванням життєвим циклом треків та навчання згорткової нейромережі детекції. Описаний рівень модульності забезпечує можливість ізольованого модифікування та тестування окремих компонентів без впливу на стабільність сусідніх підсистем, що є необхідною передумовою подальшого масштабування комплексу.

Алгоритмічне ядро системи реалізоване через три основні класи (UnscentedKalmanFilter, IMMEstimator, MerweScaledSigmaPoints) та два класи високого рівня (Track, IMMTracker), що разом утворюють завершену реалізацію гібридної архітектури взаємодіючих множинних моделей з адаптивною матрицею переходу режимів. Особлива увага була приділена коректній підтримці випадків відсутності вимірювань (метод `mark_missed`) та захисту проти числових аномалій (виродженої матриці інновації, експоненціально великих швидкостей через шум детекції), що гарантує безперебійну роботу фільтра навіть на складних відеосегментах з тривалими окклюдіями м'яча.

Геометричний модуль на основі алгоритму PnP та принципу зворотного проектування демонструє інженерну простоту та прозорість: уся калібрувальна процедура зведена до двох публічних функцій з чітко окресленими вхідними та вихідними інтерфейсами. Такий дизайн дозволяє повторно використовувати модуль для будь-якого нового запису після ручного визначення піксельних координат чотирьох реперних точок розмітки поля.

Програмний комплекс модуля навчання моделі YOLO26 використовує сучасний фреймворк Ultralytics з широким набором візуальних аугментацій, що моделюють типові спотворення волейбольного відеопотоку (рухове розмиття, шум сенсора, варіації освітлення, кольорового балансу). Завдяки цьому навчена модель демонструє підвищену стійкість до екстремальних умов реальної гри без необхідності збирати непропорційно великий ручний набір даних.

Сукупність розроблених модулів становить функціонально повний інженерний інструментарій, готовий до експериментальної верифікації та подальшого розширення додатковими підсистемами розрізання відеосегментів, автоматичного обчислення метрик якості та формування звітів. Програмний комплекс задовольняє вимогам реального часу на типовому користувацькому обладнанні і повністю відповідає теоретичним рішенням, обґрунтованим у попередніх розділах роботи.

**СИСТЕМА ПРОСТОРОВОГО АНАЛІЗУ ГРИ В ВОЛЕЙБОЛ З
ВИКОРИСТАННЯМ МАШИННОГО НАВЧАННЯ**

Принципова схема

ІАЛЦ.467200.004 Д1

ПЕРЕЛІК ФАЙЛІВ ПРОГРАМНОГО КОМПЛЕКСУ

У цьому додатку наведено повні вихідні тексти ключових модулів програмного комплексу системи трикутного просторового аналізу польоту волейбольного м'яча. Модулі впорядковано відповідно до архітектурного графа залежностей, описаного в розділі 3 пояснювальної записки: спочатку наводиться модуль геометричного зворотного проектування, далі — ядро байєсової фільтрації, потім модуль трекінгу зі скінченням автоматом життєвого циклу треку та, нарешті, сценарій навчання згорткової нейромережі-детектора. Сумарний обсяг наведеного коду становить понад 950 рядків мовою програмування Python.

ball_3d_analysis/get_court_position_methods.py

```
import cv2
import numpy as np
import torch as t
import torchvision
import torchaudio
import albumentations as A
from ultralytics import YOLO
```

```
pts_real_3d = np.array([
    [0.0, 0.0, 0.0],
    [9.0, 0.0, 0.0],
    [9.0, 0.0, 18.0],
```

					ІАЛЦ.467200.004 Д1			
Зм.	Лист	№ докум.	Підп.	Дата				
Розробив	Безщасний Р.Р.				Система просторового аналізу гри в волейбол з використанням машинного навчання		Лит.	Аркуш
Перевірів	Морозов-Леонів							36
Н. контр.							НТУУ "КПІ ім. Ігоря Сікорського", ФІОТ ІО-21	
Затвердив	Волокита А. М.							

```
[0.0, 0.0, 18.0]],dtype=np.float32)
```

```
pts_video_2d = np.array([  
    [73, 1031],  
    [1835, 1027],  
    [1504, 606],  
    [405, 606]],dtype=np.float32)
```

```
K = np.array([  
    [1300.0, 0.0, 960.0],  
    [0.0, 1300.0, 540.0],  
    [0.0, 0.0, 1.0]],dtype=np.float32)
```

```
def calibrate_camera(pts_3d, pts_2d, camera_matrix, dist):  
    success, rvec, tvec = cv2.solvePnP(pts_3d, pts_2d, camera_matrix, dist,  
    flags=cv2.SOLVEPNP_ITERATIVE)
```

```
if not success:
```

```
    raise ValueError("solvePnP не зміг знайти рішення,"  
        " треба перевірити порядок точок")
```

```
R, _ = cv2.Rodrigues(rvec)
```

```
# Знаходимо фізичну позицію камери в залі ( $C = -R^T * tvec$ )
```

```
R_inv = np.linalg.inv(R)
```

```
camera_position = -np.dot(R_inv, tvec)
```

```
return R, tvec, camera_position
```

```
def get_3d_position(u, v, bbox_w, K, R_matrix, camera_pos,  
    ball_diameter=0.21):
```

```
    """
```

Параметри:

u, v: центр BBox м'яча в пікселях

bbox_w: ширина BBox м'яча в пікселях (беремо ширину, бо вона менше
страждає від Motion Blur, ніж висота)

K: матриця камери (Intrinsic)

R_matrix: матриця обертання з solvePnP

camera_pos: 3D позиція камери (C) з solvePnP

ball_diameter: діаметр волейбольного м'яча в метрах (стандарт 21 см)

```
    """
```

```
    f_x = K[0, 0]
```

```
    c_x = K[0, 2]
```

```
    f_y = K[1, 1]
```

```
    c_y = K[1, 2]
```

```
    Z_c = (f_x * ball_diameter) / bbox_w
```

```
    x_norm = (u - c_x) / f_x
```

```
    y_norm = (v - c_y) / f_y
```

```
    d_cam = np.array([[x_norm], [y_norm], [1.0]])
```

```
    d_world = np.dot(R_matrix.T, d_cam)
```

```
    P_world = camera_pos.reshape(3, 1) + Z_c * d_world
```

```
return P_world.ravel()
```

ball_3d_analysis/IMM_UKF.py

```
import numpy as np
from copy import deepcopy
from scipy.linalg import cholesky, inv, expm, block_diag
from get_court_position_methods import calibrate_camera, get_3d_position
from scipy.stats import multivariate_normal
import sys
```

```
class UnscentedKalmanFilter(object):
```

```
    def __init__(self, name, dim_x, dim_z, dt, fx, hx, points, sqrt_fn=None,
                  x_mean_fn=None, z_mean_fn=None, residual_x=None, residual_z=None):
        self.name = name
        self.x = np.zeros(dim_x)
        self.P = np.eye(dim_x)
        self.x_prior = np.copy(self.x)
        self.P_prior = np.copy(self.P)
        self.Q = np.eye(dim_x)
        self.R = np.eye(dim_z)
        self._dim_x = dim_x
        self._dim_z = dim_z
        self.points_fn = points
        self._dt = dt
        self._num_sigmas = points.num_sigmas()
```



```

self.hx = hx
self.fx = fx
self.x_mean_fn = x_mean_fn
self.z_mean_fn = z_mean_fn
self.Wm, self.Wc = points.Wm, points.Wc
self.likelihood = None
self._log_likelihood = np.log(sys.float_info.min)
self._mahalanobis = None

if sqrt_fn is None:
    self.msqrt = cholesky
else:
    self.msqrt = sqrt_fn

if residual_x is None:
    self.residual_x = np.subtract
else:
    self.residual_x = residual_x

if residual_z is None:
    self.residual_z = np.subtract
else:
    self.residual_z = residual_z

# sigma points transformed through f(x) and h(x)
self.sigmas_f = np.zeros((self._num_sigmas, self._dim_x))
self.sigmas_h = np.zeros((self._num_sigmas, self._dim_z))

```

```

self.K = np.zeros((dim_x, dim_z))
self.y = np.zeros(dim_z)
self.z = np.array([[None]*dim_z])
self.S = np.zeros((dim_z, dim_z))
self.SI = np.zeros((dim_z, dim_z))
self.inv = np.linalg.inv

```

```

self.x_prior = self.x.copy()
self.P_prior = self.P.copy()

```

```

self.x_post = self.x.copy()
self.P_post = self.P.copy()

```

```

def predict(self, dt=None, UT=None, fx=None, **fxargs):

```

```

    """

```

```

    :param dt: time step for next iteration of computation

```

```

    :param UT: unscented transform function

```

```

    :param fx: state transition function

```

```

    :param fxargs: arguments passed to f(x)

```

```

    """

```

```

    if dt is None:

```

```

        dt = self._dt

```

```

    if UT is None:

```

```

        UT = unscented_transform

```

```

    # calculating sigma points for given mean and covariance

```

```

    self.compute_process_sigmas(dt, fx, **fxargs)

```

```

#pass sigmas through the unscented transform to compute the prior
self.x, self.P = UT(self.sigmas_f, self.Wm, self.Wc,
                    self.Q, self.x_mean_fn, self.residual_x)

self.P = (self.P + self.P.T) / 2.0

self.x_prior = np.copy(self.x)
self.P_prior = np.copy(self.P)

def update(self, z, R=None, UT=None, hx=None, **hxargs):
    """
    :param z: measurement vector
    :param R: measurement noise matrix
    :param UT: function for unscented transform
    :param hx: measurement function
    :param hxargs: arguments for hx
    """

    if z is None:
        self.z = np.array([[None]*self._dim_z]).T
        self.x_post = self.x.copy()
        self.P_post = self.P.copy()
        self.likelihood = 1.0
        return

    if hx is None:
        hx = self.hx

    if UT is None:
        UT = unscented_transform

```

```

if R is None:
    R = self.R
elif np.isscalar(R):
    R = np.eye(self._dim_z) * R

# passing prior sigmas through h(x) to get measurement sigmas
# the shape of simgas_h will vary if the shape of z varies
sigmas_h = []
for s in self.sigmas_f:
    sigmas_h.append(hx(s, **hxargs))

self.sigmas_h = np.atleast_2d(sigmas_h)

# mean and covariance of prediction passed through unscented transform
z_mean, self.S = UT(self.sigmas_h, self.Wm, self.Wc,
                    R, self.z_mean_fn, self.residual_z)
self.SI = self.inv(self.S)

# compute cross variance of the state and measurement
Pxz = self.cross_variance(self.x, z_mean, self.sigmas_f, self.sigmas_h)

self.K = np.dot(Pxz, self.SI) # Kalman gain
self.y = self.residual_z(z, z_mean)
S_stable = self.S + np.eye(self.S.shape[0]) * 1e-6
try:
    self.likelihood = multivariate_normal.pdf(self.y,
        mean=np.zeros_like(self.y), cov=S_stable)
except np.linalg.LinAlgError:

```

```

# Catching singularity effect
self.likelihood = 1e-300

# updating Gaussian state estimate(x, P)
self.x = self.x + np.dot(self.K, self.y)
self.P = self.P - np.dot(self.K, np.dot(self.S, self.K.T))

# save measurement and posterior state
self.z = deepcopy(z)
self.x_post = self.x.copy()
self.P_post = self.P.copy()

def cross_variance(self, x, z, sigmas_f, sigmas_h):
    # Computing cross variance of the state 'x' and measurement 'z'
    Pxz = np.zeros((sigmas_f.shape[1], sigmas_h.shape[1]))
    N = sigmas_f.shape[0]
    for i in range(N):
        dx = self.residual_x(sigmas_f[i], x)
        dz = self.residual_z(sigmas_h[i], z)
        Pxz += self.Wc[i] * np.outer(dx, dz)
    return Pxz

def compute_process_sigmas(self, dt, fx=None, **fx_args):
    # computes the values of sigmas_f
    if fx is None:
        fx = self.fx

    # calculate sigma points for given mean and covariance

```

```
sigmas = self.points_fn.sigma_points(self.x, self.P)
```

```
for i, s in enumerate(sigmas):
```

```
    self.sigmas_f[i] = fx(s, dt, **fx_args)
```

```
@property
```

```
def log_likelihood(self):
```

```
    """
```

```
    log-likelihood of the last measurement.
```

```
    """
```

```
    if self._log_likelihood is None:
```

```
        self._log_likelihood = logpdf(x=self.y, cov=self.S)
```

```
    return self._log_likelihood
```

```
@property
```

```
def mahalanobis(self):
```

```
    """
```

```
    Mahalanobis distance of measurement. E.g. 3 means measurement
    was 3 standard deviations away from the predicted value.
```

```
    """
```

```
    if self._mahalanobis is None:
```

```
        self._mahalanobis = np.sqrt(float(np.dot(np.dot(self.y.T, self.SI), self.y)))
```

```
    return self._mahalanobis
```

```
class IMMEstimator(object):
```

```
    """ Implements an Interacting Multiple-Model (IMM) estimator.
```

```
    :param filters: N-list consisting of filters in sequenced order
```

```
:param mu: (N) array-like of float with mode probabilities for each filter
:param M: (N, N) Markov chain transition matrix
''''''
```

```
def __init__(self, filters, mu, M):
    if len(filters) < 2:
        raise ValueError('filters must contain at least two filters')

    self.filters = filters
    self.mu = np.asarray(mu) / np.sum(mu)
    self.M = M

    x_shape = filters[0].x.shape
    for f in filters:
        if x_shape != f.x.shape:
            raise ValueError(
                'All filters must have the same state dimension')

    self.x = np.zeros(filters[0].x.shape)
    self.P = np.zeros(filters[0].P.shape)
    self.N = len(filters)
    self.likelihood = np.zeros(self.N)
    self.omega = np.zeros((self.N, self.N))
    self._compute_mixing_probabilities()

    # initialize imm state estimate based on current filters
    self._compute_state_estimate()
    self.x_prior = self.x.copy()
```

```

self.P_prior = self.P.copy()
self.x_post = self.x.copy()
self.P_post = self.P.copy()
self._log_likelihood = np.log(sys.float_info.min)
self._mahalanobis = None

def filter_setx(self):
    for f in self.filters:
        f.x = self.x

def update(self, z):
    """
    Add a new measurement (z) to the Kalman filter. If z is None, nothing
    is changed.
    :param z: measurement
    """

    # run update on each filter, and save the likelihood
    for i, f in enumerate(self.filters):
        f.update(z)
        self.likelihood[i] = f.likelihood

    # update mode probabilities from total probability * likelihood
    self.mu = self.cbar * self.likelihood
    sum_mu = np.sum(self.mu)
    if sum_mu == 0.0:
        self.mu = self.cbar.copy()
    else:

```



```

self.mu /= sum_mu

self._compute_mixing_probabilities()

# compute mixed IMM state and covariance and save posterior estimate
self._compute_state_estimate()
self.x_post = self.x.copy()
self.P_post = self.P.copy()

def predict(self, u=None):
    """
    Predict next state (prior) using the IMM state propagation
    equations.
    :param u: control vector, if None then none
    """

    # compute mixed initial conditions
    xs, Ps = [], []
    for i, (f, w) in enumerate(zip(self.filters, self.omega.T)):
        x = np.zeros(self.x.shape)
        for kf, wj in zip(self.filters, w):
            x += kf.x * wj
        xs.append(x)

        P = np.zeros(self.P.shape)
        for kf, wj in zip(self.filters, w):
            y = kf.x - x
            P += wj * (np.outer(y, y) + kf.P)

```

```

        Ps.append(P)

# compute each filter's prior using the mixed initial conditions
for i, f in enumerate(self.filters):
    # propagate using the mixed state estimate and covariance
    f.x = xs[i].copy()
    f.P = Ps[i].copy()
    f.predict(u)

# compute mixed IMM state and covariance and save posterior estimate
self._compute_state_estimate()
self.x_prior = self.x.copy()
self.P_prior = self.P.copy()

def _compute_state_estimate(self):
    """
    Computes the IMM's mixed state estimate from each filter using
    the the mode probability self.mu to weight the estimates.
    """
    self.x.fill(0)
    for f, mu in zip(self.filters, self.mu):
        self.x += f.x * mu

    self.P.fill(0)
    for f, mu in zip(self.filters, self.mu):
        y = f.x - self.x
        self.P += mu * (np.outer(y, y) + f.P)

```

```

def _compute_mixing_probabilities(self):
    # Compute the mixing probability for each filter

    self.cbar = np.dot(self.mu, self.M)
    for i in range(self.N):
        for j in range(self.N):
            self.omega[i, j] = (self.M[i, j]*self.mu[i]) / self.cbar[j]

@property
def log_likelihood(self):
    """
    log-likelihood of the last measurement.
    """
    if self._log_likelihood is None:
        self._log_likelihood = logpdf(x=self.y, cov=self.S)
    return self._log_likelihood

@property
def mahalanobis(self):
    """
    Mahalanobis distance of measurement. E.g. 3 means measurement
    was 3 standard deviations away from the predicted value.
    """
    if self._mahalanobis is None:
        self._mahalanobis = np.sqrt(float(np.dot(np.dot(self.y.T, self.SI), self.y)))
    return self._mahalanobis

```

class MerweScaledSigmaPoints(object):

"""

Generates sigma points and weights according to Van der Merwe's 2004 dissertation[1] for the UnscentedKalmanFilter class. It parametrizes the sigma points using alpha, beta, kappa terms, and is the version seen in most publications.

:param n : int

Dimensionality of the state. $2n+1$ weights will be generated.

:param alpha : float

Determines the spread of the sigma points around the mean.

Usually a small positive value ($1e-3$).

:param beta : float

Incorporates prior knowledge of the distribution of the mean. For Gaussian α beta=2 is optimal.

:param kappa : float, default=0.0

Secondary scaling parameter usually set to 0 or to $3-n$.

:param sqrt_method : function(ndarray), default=scipy.linalg.cholesky

Defines how we compute the square root of a matrix, which has no unique answer. Cholesky is the default choice due to its speed(another good choice - scipy.linalg.sqrtn).

:param subtract : callable (x, y), optional

Function that computes the difference between x and y.

You will have to supply this if your state variable cannot support subtraction, such as angles (359-1 degrees is 2, not 358). x and y are state vectors, not scalars.

Attributes:

Wm : np.array

weight for each sigma point for the mean

Wc : np.array

weight for each sigma point for the covariance

""""

```
def __init__(self, n, alpha, beta, kappa, sqrt_method=None, subtract=None):
```

```
    #pylint: disable=too-many-arguments
```

```
    self.n = n
```

```
    self.alpha = alpha
```

```
    self.beta = beta
```

```
    self.kappa = kappa
```

```
    if sqrt_method is None:
```

```
        self.sqrt = cholesky
```

```
    else:
```

```
        self.sqrt = sqrt_method
```

```
    if subtract is None:
```

```
        self.subtract = np.subtract
```

else:

self.subtract = subtract

self._compute_weights()

def num_sigmas(self):

""" Number of sigma points for each variable in the state x """

return 2*self.n + 1

def sigma_points(self, x, P):

""" Computes the sigma points for an unscented Kalman filter
given the mean (x) and covariance(P) of the filter.

Returns tuple of the sigma points and weights.

Works with both scalar and array inputs:

sigma_points (5, 9, 2) # mean 5, covariance 9

sigma_points ([5, 2], 9*eye(2), 2) # means 5 and 2, covariance 9I

:param x : An array-like object of the means of length n

:param P : scalar, or np.array

Covariance of the filter. If scalar, is treated as eye(n)*P.

:return np.array, of size (n, 2n+1)

Two dimensional array of sigma points. Each column contains all of
the sigmas for one dimension in the problem space.

Ordered by X_{i_0} , $X_{i_{\{1..n\}}}$, $X_{i_{\{n+1..2n\}}}$

''''''

```
if self.n != np.size(x):
    raise ValueError("expected size(x) {}, but size is {}".format(
        self.n, np.size(x)))
```

```
n = self.n
```

```
if np.isscalar(x):
    x = np.asarray([x])
```

```
if np.isscalar(P):
    P = np.eye(n)*P
else:
    P = np.atleast_2d(P)
```

```
lambda_ = self.alpha**2 * (n + self.kappa) - n
U = self.sqrt((lambda_ + n)*P)
```

```
sigmas = np.zeros((2*n+1, n))
sigmas[0] = x
```

```
for k in range(n):
    # pylint: disable=bad-whitespace
    sigmas[k+1] = self.subtract(x, -U[k])
    sigmas[n+k+1] = self.subtract(x, U[k])
```

```
return sigmas
```

```
def _compute_weights(self):
```

```
    """ Computes the weights for the scaled unscented Kalman filter. """
```

```
    n = self.n
```

```
    lambda_ = self.alpha**2 * (n + self.kappa) - n
```

```
    c = .5 / (n + lambda_)
```

```
    self.Wc = np.full(2*n + 1, c)
```

```
    self.Wm = np.full(2*n + 1, c)
```

```
    self.Wc[0] = lambda_ / (n + lambda_) + (1 - self.alpha**2 + self.beta)
```

```
    self.Wm[0] = lambda_ / (n + lambda_)
```

```
def __repr__(self):
```

```
    return (f'MerweScaledSigmaPoints(n={self.n}, alpha={self.alpha}, '
            f'beta={self.beta}, kappa={self.kappa})')
```

```
def unscented_transform(sigmas, Wm, Wc, noise_cov=None,
                        mean_fn=None, residual_fn=None):
```

```
    """
```

```
    Computes unscented transform of a set of sigma points and weights.
```

```
    returns the mean and covariance in a tuple.
```

```
    :param residual_fn:
```

```
    :param sigmas: ndarray, of size (n, 2n+1)
```

					ИАЛЦ.467200.004 Д1	Архив
						20
Зм.	Лист	№ докум.	Підп.	Дата		

2D array of sigma points.

:param Wm : ndarray [# sigmas per dimension]

Weights for the mean.

:param Wc : ndarray [# sigmas per dimension]

Weights for the covariance.

:param noise_cov: ndarray, optional

noise matrix added to the final computed covariance matrix.

:param mean_fn: callable (sigma_points, weights), optional

Function that computes the mean of the provided sigma points
and weights

''''''

kmax, n = sigmas.shape

try:

if mean_fn is None:

new mean is just the sum of the sigmas * weight

x = np.dot(Wm, sigmas) # dot = $\sum (W[k] * X_i[k])$

else:

x = mean_fn(sigmas, Wm)

except:

print(sigmas)

raise

```
# new covariance is the sum of the outer product of the residuals
# times the weights
```

```
# this is the fast way to do this - see 'else' for the slow way
if residual_fn is np.subtract or residual_fn is None:
```

```
    y = sigmas - x[np.newaxis, :]
    P = np.dot(y.T, np.dot(np.diag(Wc), y))
```

```
else:
```

```
    P = np.zeros((n, n))
    for k in range(kmax):
        y = residual_fn(sigmas[k], x)
        P += Wc[k] * np.outer(y, y)
```

```
if noise_cov is not None:
```

```
    P += noise_cov
```

```
return x, P
```

```
def Q_discrete_white_noise(dim, dt=1., var=1., block_size=1, order_by_dim=True):
    """
```

Returns the Q matrix for the Discrete Constant White Noise Model. dim may be either 2, 3, or 4 dt is the time step, and sigma is the variance in the noise.

Q is computed as the $G * G^T * \text{variance}$, where G is the process noise per

time step. In other words, $G = [[.5dt^2][dt]]^T$ for the constant velocity model.

:param dim : int (2, 3, or 4)

dimension for Q, where the final dimension is (dim x dim)

:param dt : float, default=1.0

time step in whatever units your filter is using for time. i.e. the amount of time between innovations

:param var : float, default=1.0

variance in the noise

:param block_size : int >= 1

If your state variable contains more than one dimension, such as a 3d constant velocity model $[x \ x' \ y \ y' \ z \ z']^T$, then Q must be a block diagonal matrix.

:param order_by_dim : bool, default=True

Defines ordering of variables in the state vector. 'True' orders by keeping all derivatives of each dimensions)

$[x \ x' \ x'' \ y \ y' \ y'']$

whereas 'False' interleaves the dimensions

$[x \ y \ z \ x' \ y' \ z' \ x'' \ y'' \ z'']$

''''''

if not (dim == 2 or dim == 3 or dim == 4):

raise ValueError("dim must be between 2 and 4")

```

if dim == 2:
    Q = [[.25*dt**4, .5*dt**3],
          [.5*dt**3, dt**2]]
elif dim == 3:
    Q = [[.25*dt**4, .5*dt**3, .5*dt**2],
          [.5*dt**3, dt**2, dt],
          [.5*dt**2, dt, 1]]
else:
    Q = [(dt**6)/36, (dt**5)/12, (dt**4)/6, (dt**3)/6],
          [(dt**5)/12, (dt**4)/4, (dt**3)/2, (dt**2)/2],
          [(dt**4)/6, (dt**3)/2, dt**2, dt],
          [(dt**3)/6, (dt**2)/2, dt, 1.]]

if order_by_dim:
    return block_diag(*[Q]*block_size) * var
return order_by_derivative(np.array(Q), dim, block_size) * var

def logpdf(x, mean=None, cov=1, allow_singular=True):
    """
    Computes the log of the probability density function of the normal
    N(mean, cov) for the data x. The normal may be univariate or multivariate.
    """

    if mean is not None:
        flat_mean = np.asarray(mean).flatten()
    else:
        flat_mean = None

```

```
flat_x = np.asarray(x).flatten()
```

```
return multivariate_normal.logpdf(flat_x, flat_mean,  
                                   cov, allow_singular=allow_singular)
```

```
def fx_ballistic(x, dt):
```

```
    """Балістична нелінійна модель:  $x_{next} = F \cdot x + gravity\_effect$ 
```

```
    F-матриця для  $[x, vx, y, vy, z, vz]$ """
```

```
    vx, vy, vz = x[1], x[3], x[5]
```

```
    vx = np.clip(vx, -40.0, 40.0)
```

```
    vy = np.clip(vy, -40.0, 40.0)
```

```
    vz = np.clip(vz, -40.0, 40.0)
```

```
    F = np.array([[1, dt, 0, 0, 0, 0],           # x  
                  [0, 1, 0, 0, 0, 0],           # vx  
                  [0, 0, 1, dt, 0, 0],           # y  
                  [0, 0, 0, 1, 0, 0],           # vy  
                  [0, 0, 0, 0, 1, dt],           # z  
                  [0, 0, 0, 0, 0, 1]], dtype=float) # vz
```

```
    B = np.diag([0.5 * dt**2, dt, 0.5 * dt**2, dt, 0.5 * dt**2, dt])
```

```
    k = 0.0314
```

```
    v_mag = np.sqrt(x[1]**2 + x[3]**2 + x[5]**2)
```

```
    a_dx = -k * v_mag * vx
```

```
    a_dy = -k * v_mag * vy
```

```
    a_dz = -k * v_mag * vz
```

```

u = np.array([a_dx, a_dx, a_dy - 9.81, a_dy - 9.81, a_dz, a_dz])

x_next = np.dot(F, x) + np.dot(B, u)

return x_next

def fx_hit(x, dt):
    """
    Модель удару: Constant Velocity.
    Стан: [x, vx, y, vy, z, vz]
    """
    F = np.array([[1, dt, 0, 0, 0, 0],
                  [0, 1, 0, 0, 0, 0],
                  [0, 0, 1, dt, 0, 0],
                  [0, 0, 0, 1, 0, 0],
                  [0, 0, 0, 0, 1, dt],
                  [0, 0, 0, 0, 0, 1]], dtype=float)

    return np.dot(F, x)

def fx_bounce(x, dt):
    """Модель відскоку, марковська матриця"""
    epsilon = 0.75 # Коефіцієнт реституції
    friction = 0.85 # Коефіцієнт тертя
    x_next = np.copy(x)
    vx_new = x[1] * friction
    vy_new = -x[3] * epsilon
    vz_new = x[5] * friction

```

```

x_next[0] += vx_new * dt
x_next[2] += vy_new * dt
x_next[4] += vz_new * dt

```

```

x_next[1] = vx_new
x_next[3] = vy_new
x_next[5] = vz_new

```

```

return x_next

```

```

def hx(x):
    return np.array([x[0], x[2], x[4]])

```

```

def measurement_transform(prediction_data, K, R_matrix, camera_pos,
                           ball_diameter=0.21):
    u_cam, v_cam, w_cam = (prediction_data['x_pos'], prediction_data['y_pos'],
                           prediction_data['w_box'])
    raw_x, raw_y, raw_z = get_3d_position(u_cam, v_cam, w_cam, K,
                                           R_matrix, camera_pos, ball_diameter)
    raw_y = -raw_y
    measurement = np.array([raw_x, raw_y, raw_z], dtype=np.float32)

    return measurement

```

```

def get_dynamic_transition_matrix(y, vy):
    """

```

Повертає марковську матрицю 3x3 для моделей: [Ballistic, Hit, Bounce]

''''''

```
M = np.array([[0.95, 0.04, 0.01],  
              [0.60, 0.40, 0.00],  
              [0.90, 0.00, 0.10]])
```

```
if y < 0.3 and vy < 0:  
    M[0] = [0.10, 0.05, 0.85]  
    M[1] = [0.10, 0.05, 0.85]
```

```
return M
```

ball_3d_analysis/IMMTracker.py

```
import numpy as np  
from scipy.optimize import linear_sum_assignment  
from IMM_UKF import *  
import cv2
```

```
def create_imm_estimator(z_initial, dt=0.02):  
    # dt is forwarded from IMMTracker (1 / source FPS);  
    # default corresponds to 50 FPS  
    dim_x = 6  
    dim_z = 3  
    points = MerweScaledSigmaPoints(n=dim_x, alpha=.1, beta=2., kappa=1.)  
    P_init = np.diag([0.1, 50.0, 0.1, 50.0, 0.1, 50.0])  
    R_init = np.diag([0.01, 0.01, 0.04])
```



```

# Для балістичної моделі Q мінімальна
q_var_ballistic = 0.1
q_b = Q_discrete_white_noise(dim=2, dt=dt, var=q_var_ballistic)
Q_ballistic = block_diag(q_b, q_b, q_b)

# Для моделі удару Q велика
q_var_hit = 100
q_h = Q_discrete_white_noise(dim=2, dt=dt, var=q_var_hit)
Q_hit = block_diag(q_h, q_h, q_h)

# Для відскоку Q середня
q_var_bounce = 5
q_bnc = Q_discrete_white_noise(dim=2, dt=dt, var=q_var_bounce)
Q_bounce = block_diag(q_bnc, q_bnc, q_bnc)

# Ballistic filter
ukf_ballistic = UnscentedKalmanFilter(name='ballistic UKF', dim_x=dim_x,
                                       dim_z=dim_z, dt=dt, fx=fx_ballistic, hx=hx, points=points)
ukf_ballistic.P = P_init
ukf_ballistic.Q = Q_ballistic
ukf_ballistic.R = R_init

# Hit filter
ukf_hit = UnscentedKalmanFilter(name='hit UKF', dim_x=dim_x, dim_z=dim_z,
                                dt=dt, fx=fx_hit, hx=hx, points=points)
ukf_hit.P = P_init
ukf_hit.Q = Q_hit
ukf_hit.R = R_init

```

```
# Bounce filter
ukf_bounce = UnscentedKalmanFilter(name='bounce UKF', dim_x=dim_x,
                                     dim_z=dim_z, dt=dt, fx=fx_bounce, hx=hx, points=points)
ukf_bounce.P = P_init
ukf_bounce.Q = Q_bounce
ukf_bounce.R = R_init
```

```
filters_lt = [ukf_ballistic, ukf_hit, ukf_bounce]
mu = np.array([0.95, 0.04, 0.01])
M_base = np.array([[0.95, 0.04, 0.01],
                   [0.60, 0.40, 0.00],
                   [0.90, 0.00, 0.10]])
```

```
imm = IMMEstimator(filters_lt, mu, M_base)
```

```
# Ініціалізуємо стан
initial_x = np.array([z_initial[0], 0.0, z_initial[1], 0.0, z_initial[2], 0.0])
for f in imm.filters:
    f.x = initial_x.copy()
imm._compute_state_estimate()
```

```
return imm
```

```
class Track:
```

```
def __init__(self, track_id, z_initial, dt):
    self.track_id = track_id
    self.imm = create_imm_estimator(z_initial, dt)
```

```

# ЖИТТЄВИЙ ЦИКЛ

self.state = 'Tentative' # Можливі: 'Tentative', 'Confirmed', 'Deleted'

self.time_since_update = 0

self.hits = 1

self.hit_streak = 1

self.h_matrix = np.array([
    [1, 0, 0, 0, 0, 0],
    [0, 0, 1, 0, 0, 0],
    [0, 0, 0, 0, 1, 0]
], dtype=float)

def predict(self):
    self.imm.M = get_dynamic_transition_matrix(self.imm.x[2], self.imm.x[3])
    self.imm.predict()
    self.time_since_update += 1

def update(self, z):
    self.imm.update(z)
    self.time_since_update = 0
    self.hits += 1
    self.hit_streak += 1
    if self.state == 'Tentative' and self.hits >= 3:
        self.state = 'Confirmed'

def mark_missed(self):
    self.hit_streak = 0
    self.imm.update(None) # Екстраполяція без вимірювання

```

```

def get_mahalanobis_distance(self, z, R_matrix):
    """
    Обчислює відстань Махаланобіса між прогнозом IMM та
    новою детекцією.
    """
    # Проектуємо змішаний стан x_prior та коваріацію
    # P_prior у простір вимірювань
    z_mean = np.dot(self.h_matrix, self.imm.x_prior)
    S = np.dot(self.h_matrix, np.dot(self.imm.P_prior,
                                     self.h_matrix.T)) + R_matrix

    y = z - z_mean # Вектор невязки

    try:
        S_inv = np.linalg.inv(S)
        dist_sq = np.dot(y.T, np.dot(S_inv, y))
        return dist_sq
    except np.linalg.LinAlgError:
        return 1e5

```

```

class IMMTracker:

```

```

    def __init__(self, dt=0.02, max_age=50, min_hits=3, gating_threshold=11.34):
        self.dt = dt

        self.max_age = max_age # К-сть кадрів до видалення треку
        self.min_hits = min_hits # К-сть кадрів для підтвердження
        self.gating_threshold = gating_threshold # Хи-квадрат поріг

```

```

self.tracks = []
self.next_id = 1
self.R_matrix = np.diag([0.01, 0.01, 0.04]) # Базова матриця похибки

```

```

def update(self, detections_3d):

```

```

    """

```

```

    detections_3d: список масивів np.array([x, y, z]) для
    всіх знайдених об'єктів у кадрі

```

```

    """

```

```

    for track in self.tracks:

```

```

        track.predict()

```

```

    if len(detections_3d) == 0:

```

```

        for track in self.tracks:

```

```

            track.mark_missed()

```

```

        self._manage_lifecycle()

```

```

        return

```

```

    if len(self.tracks) == 0:

```

```

        for z in detections_3d:

```

```

            self.tracks.append(Track(self.next_id, z, self.dt))

```

```

            self.next_id += 1

```

```

        return

```

```

    cost_matrix = np.full((len(self.tracks), len(detections_3d)), 1e5)

```

```

    for t, track in enumerate(self.tracks):

```

```

for d, z in enumerate(detections_3d):
    dist_sq = track.get_mahalanobis_distance(z, self.R_matrix)
    if dist_sq < self.gating_threshold:
        cost_matrix[t, d] = dist_sq

```

3. Асоціація (Угорський алгоритм)

```

row_ind, col_ind = linear_sum_assignment(cost_matrix)

```

```

unmatched_tracks = set(range(len(self.tracks)))

```

```

unmatched_detections = set(range(len(detections_3d)))

```

4. Оновлення знайдених пар

```

for r, c in zip(row_ind, col_ind):
    if cost_matrix[r, c] < self.gating_threshold:
        self.tracks[r].update(detections_3d[c])
        unmatched_tracks.remove(r)
        unmatched_detections.remove(c)

```

5. Обробка пропущених треків

```

for t in unmatched_tracks:
    self.tracks[t].mark_missed()

```

6. Створення нових треків для нерозпізнаних детекцій

```

for d in unmatched_detections:
    self.tracks.append(Track(self.next_id, detections_3d[d], self.dt))
    self.next_id += 1

```

7. Видалення старих треків

```
self._manage_lifecycle()
```

```
def _manage_lifecycle(self):
```

```
    active_tracks = []
```

```
    for track in self.tracks:
```

```
        # Видаляємо трек, якщо він довго не оновлювався
```

```
        if track.time_since_update > self.max_age:
```

```
            track.state = 'Deleted'
```

```
        # Трек залишається, якщо він не видалений
```

```
        if track.state != 'Deleted':
```

```
            active_tracks.append(track)
```

```
    self.tracks = active_tracks
```

```
def get_confirmed_tracks(self):
```

```
    """Повертає позиції лише тих треків, в яких ми впевнені"""
```

```
    return [t for t in self.tracks if t.state == 'Confirmed']
```

training_models/model_train_remote.py

```
import os
```

```
import torch as t
```

```
import torch.nn as nn
```

```
import torch.nn.functional as F
```

```
from pathlib import Path
```

```
from torchvision import transforms
```

```
from torchvision.transforms import Compose, ToTensor, PILToTensor, Resize
```

```
from tqdm import tqdm
```

```

import numpy as np
# import optuna as opt
from ultralytics import YOLO
import albumentations as A

main_model = YOLO(' ../parsing_datasets_scripts/yolo26m.pt')

custom_transforms = [
    A.MotionBlur(blur_limit=(7, 15), p=0.4),
    A.GaussNoise(std_range=(0.01, 0.05), p=0.2),
    A.CLAHE(clip_limit=4.0, p=0.3),
    A.RandomBrightnessContrast(brightness_limit=0.25,
                               contrast_limit=0.25, p=0.4),
    A.HueSaturationValue(hue_shift_limit=15, sat_shift_limit=30,
                         val_shift_limit=20, p=0.3),
]

# Purely for server training
main_model.train(data='/workspace/diploma_project/data/server_data.yaml',
                 epochs=100, device='0', imgsz=640,
                 augmentations=custom_transforms, patience=20)

metrics = main_model.val(data='/workspace/diploma_project/data'
                        '/server_data.yaml', imgsz=640)

print(f'mAP50: {metrics.box.map50}')
print(f'mAP50-95: {metrics.box.map}')

```